

TRƯỜNG ĐẠI HỌC PHENIKAA
KHOA ĐIỆN – ĐIỆN TỬ



BÁO CÁO CUỐI KỲ
LẬP TRÌNH ỨNG DỤNG FPGA

**Tên đề tài: Thiết kế một bộ đo nhiệt độ, độ ẩm và áp suất trên FPGA –
Measuring Device**

Sinh viên thực hiện: Nguyễn Tuấn Hiệp MSSV: 21012789 Lớp: K15 – DTVT1

Giảng viên hướng dẫn: TS. Lê Văn Thuận
Khoa Điện – Điện tử

Hà Nội, Học kỳ I, năm học 2025-2026

TRƯỜNG ĐẠI HỌC PHENIKAA
KHOA ĐIỆN – ĐIỆN TỬ



BÁO CÁO CUỐI KỲ
LẬP TRÌNH ỨNG DỤNG FPGA

TÊN ĐỀ TÀI:

Thiết kế một bộ đo nhiệt độ, độ ẩm và áp suất trên FPGA – Measuring Device

Sinh viên thực hiện: Bùi Minh Thắng MSSV: 21013335 Lớp: K15 – DTVT2
Nguyễn Văn Dũng MSSV: 21013348 Lớp: K15 – DTVT2
Nguyễn Tuấn Hiệp MSSV: 21012789 Lớp: K15 – DTVT1

Giảng viên hướng dẫn: TS. Lê Văn Thuận Khoa Điện – Điện tử

Hà Nội, Học kỳ I, năm học 2025-2026

LỜI CAM ĐOAN

Chúng tôi xin cam đoan rằng đề tài **“Thiết kế một bộ đo nhiệt độ, độ ẩm và áp suất trên FPGA – Measuring Device”** là một công trình nghiên cứu độc lập, do chính chúng tôi thực hiện trong quá trình học tập và nghiên cứu tại trường. Đề tài không có sự sao chép, thuê mượn, hay sử dụng bất kỳ phần nào từ các công trình của người khác mà không được phép.

Trong quá trình thực hiện, chúng tôi đã tham khảo một số tài liệu, nguồn thông tin có nguồn gốc rõ ràng và được trích dẫn đầy đủ theo quy định. Đồng thời, chúng tôi đã nhận được sự hướng dẫn tận tình và hỗ trợ quý báu từ *TS. Lê Văn Thuận – Giảng viên Khoa Điện – Điện tử*, Trường Đại học Phenikaa.

Chúng tôi xin cam đoan sẽ chịu hoàn toàn trách nhiệm về tính trung thực và độc lập của công trình nghiên cứu này. Nếu có bất kỳ vấn đề nào phát sinh liên quan đến tính nguyên gốc của đề tài, chúng tôi sẵn sàng chấp nhận mọi hình thức xử lý theo quy định của nhà trường và pháp luật.

Xin trân trọng cảm ơn!

PHIẾU ĐÁNH GIÁ BÁO CÁO CUỐI KỲ
HỌC PHẦN LẬP TRÌNH ỨNG DỤNG FPGA

Rubric 4: Bài tập lớn cuối kỳ

Mức độ đạt chuẩn quy định						Trọng số	Điểm
Tiêu chí đánh giá	Điểm (0-3.9)	Điểm (4.0-5.4)	Điểm (5.5-6.9)	Điểm (7.0-8.4)	Điểm (8.5-10)		
Chất lượng sản phẩm	Đáp ứng dưới 40% yêu cầu	Đáp ứng dưới 50% yêu cầu	Đáp ứng 50%- 70% yêu cầu	Đáp ứng 70%- 80% yêu cầu	Đáp ứng 80%- 100% yêu cầu	50%	
Hình thức/nội dung báo cáo	Đơn điệu, chữ nhỏ, nhiều lỗi chính tả	Chữ nhỏ, có lỗi chính tả	Chữ nhỏ, có rất ít lỗi chính tả	Chữ to, rõ ràng, có lỗi chính tả	Đẹp, rõ, không lỗi chính tả	10% Slides	
	Trình bày một cách cầu thả, copy từ bài khác, nội dung sơ sài không đáp ứng đủ cho 1 quyển báo cáo	Trình bày nội dung sơ sài, lỗi chính tả nhiều	Trình bày còn thiếu ý, nội dung chưa đầy đủ, nhiều lỗi trình bày và chính tả	Trình bày nội dung 1 cách rõ ràng, có một số lỗi trình bày, chính tả	Trình bày rõ ràng đầy đủ nội dung, bố cục của 1 quyển báo cáo. Ít có lỗi chính tả	10% Quyển báo cáo	
Kỹ năng nhóm	< 20% thành viên tham gia thực hiện/trình bày	~ 40% thành viên tham gia thực hiện/trình bày	~ 60% thành viên tham gia thực hiện/trình bày	~ 80% thành viên tham gia thực hiện/trình bày	100% thành viên tham gia thực hiện/trình bày	10%	
Kỹ năng trình bày	Nói nhỏ, không tự tin, không giao lưu người nghe	Nói nhỏ, không tự tin, đọc slide	Nói to, trình bày ngập ngừng, có giao lưu với người nghe	Nói rõ, không tự tin, trình bày chưa lưu loát, giao lưu	Nói rõ, tự tin, thuyết phục, giao lưu người nghe	10%	

Mức độ đạt chuẩn quy định						Trọng số	Điểm
Tiêu chí đánh giá	Điểm (0-3.9)	Điểm (4.0-5.4)	Điểm (5.5-6.9)	Điểm (7.0-8.4)	Điểm (8.5-10)		
				với người nghe			
Trả lời câu hỏi	Trả lời đúng dưới 1/3 số câu hỏi	Trả lời đúng dưới 1/2	Trả lời đúng trên 1/2 số câu hỏi	Trả lời đúng trên 2/3 số câu hỏi	Trả lời đúng tất cả các câu hỏi	10%	
Điểm tổng							

Lưu ý: Tiêu chí chất lượng sản phẩm và quan trọng nhất. Báo cáo, sản phẩm phải do bản thân sinh viên/nhóm sinh viên thực hiện. Nếu có dấu hiệu copy, thuê làm Đồ án hoặc đạt được sản phẩm nhưng không nắm được nội dung cũng như kết quả thì sinh viên bị 0 điểm. *Mọi tiêu chí đánh giá khác chỉ được xem xét khi sinh viên đạt được yêu cầu của tiêu chí này.*

Cán bộ chấm thi 1
(Ký và ghi rõ họ tên)

Cán bộ chấm thi 2
(Ký và ghi rõ họ tên)

TIẾN ĐỘ THỰC HIỆN

I) Thành viên nhóm

STT	Hình Ảnh	Thông tin cá nhân
1		- Họ và tên: Bùi Minh Thắng - Mã SV: 21013335 - Lớp: K15-ĐTVT2 - Trường Đại học Phenikaa. - Nơi ở: Nam Định
2		- Họ và tên: Nguyễn Văn Dũng - Mã SV: 21013348 - Lớp: K15-ĐTVT2 - Trường Đại học Phenikaa. - Nơi ở: Hà Nội
3		- Họ và tên: Nguyễn Tuấn Hiệp - Mã SV: 21012789 - Lớp: K15-ĐTVT1 - Trường Đại học Phenikaa. - Nơi ở: Nam Định

II) Phân công nhiệm vụ

Thành viên	Phân công nhiệm vụ	Hoàn thành
Bùi Minh Thắng	Hiện thị lên LCD, viết báo cáo	Đạt
Nguyễn Văn Dũng	Đo dữ liệu DHT11, Thiết kế mạch điện, viết báo cáo	Đạt
Nguyễn Tuấn Hiệp	Hiện thị LCD ,slide thuyết trình	Đạt

II) Tiến độ thực hiện

Nội dung	Thời gian thực hiện	Kết quả
Tìm hiểu đề tài, đọc tài liệu tham khảo, nguyên lí hoạt động.	15/2/2025-19/2/2025	Hoàn thành
Tìm hiểu các chức năng.	20/2/2025-25/2/2025	Hoàn thành

Tiến hành chạy code demo.	25/2/2025-10/3/2025	Hoàn thành
Code hoàn thiện,làm slide,báo cáo.	10/3/2025-25/3/2025	Hoàn thành
Xem lại toàn bộ quá trình.	25/3/2025-27/3/2025	Hoàn thành

Sinh viên thực hiện
(Ký, ghi rõ họ tên)

MỤC LỤC

DANH MỤC BẢNG BIỂU	9
LỜI NÓI ĐẦU	11
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	15
1. Ngôn ngữ lập trình phần cứng Verilog HDL.....	15
2. Mạch tích hợp logic có thể lập trình được (FPGA).....	16
3. Phần mềm lập trình Quartus II	19
4 Testbench.....	21
5 Kit De0 Nano.....	22
6. LCD 16x2.....	26
7 Các loại mô hình Verilog	32
❖Structural Verilog	32
❖ Behavioral Verilog.....	34
❖Dataflow Verilog.....	35
8Các loại máy trạng thái FSM	36
<i>Máy trạng thái Mealy</i>	36
❖ <i>Máy trạng thái Moore.</i>	37
9Tổng kết chương	39
CHƯƠNG 2: THIẾT KẾ VÀ XÂY DỰNG MÔ HÌNH HỆ THỐNG	40
1. Mô hình hệ thống.....	40
1.1 Sơ đồ hệ thống.....	40
1.2 Nguyên lý hoạt động	40
1.3.. Nguyên lý hoạt động của Top_Module.....	62
2. Tổng kết chương	62
CHƯƠNG 3: GIẢI THÍCH VÀ KẾT QUẢ.....	67
1. Kết quả thực nghiệm	67
2. Kết luận	67
KẾT LUẬN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN	69
TÀI LIỆU THAM KHẢO.....	71

DANH MỤC BẢNG BIỂU

Bảng 1 : Bảng gán chân GPIO.....	26
Bảng 2 : Bảng lệnh điều khiển LCD.....	28
Bảng 3: Bảng mã ASCII.....	29

DANH MỤC HÌNH ẢNH

Hình 1: Ví dụ về HDL.....	16
Hình 2 :Cấu trúc FPGA.....	17
Hình 3: Ví dụ về FPGA.....	19
Hình 4: Giao diện phần mềm lập trình Quartus II 13.1	21
Hình 5 Kit DeO Nano.....	22
Hình 6 Cấu trúc của Kit DeO Nano.....	23
Hình 7:Sơ đồ chân PIN của GPIO.....	25
Hình 8 : Bảng gán chân GPIO.....	28
Hình 9:Cảm biến nhiệt độ độ ẩm.....	29
Hình 10: Bộ cộng bán phần.....	32
Hình 11: Bộ cộng toàn phần.....	32
Hình12:Máy trạng thái Mealy.....	33
Hình 13: Máy trạng thái Moore	39
Hình 14: Mô hình hệ thống.....	39
Hình 15: State machine DHT11.....	38
Hình 16: State machine LCD.....	39
Hình 17: Sơ đồ nguyên lý.....	40
Hình 18: Lưu đồ thuật toán module Clock_1MHz.....	57
Hình 19: Lưu đồ thuật toán LCD.....	61
Hình 20:: Lưu đồ thuật toán.....	66

LỜI NÓI ĐẦU

Lý do chọn đề tài

Ngày nay, với sự đòi hỏi của sự phát triển Khoa học – Kỹ thuật, thì các thiết bị đo lường ngày càng xuất hiện nhiều và có ảnh hưởng đáng kể đến các lĩnh vực trong kinh tế và công nghiệp. Vì vậy một thiết bị đo số liệu đa năng có thể đo được các thông số của nhiệt độ, độ ẩm, áp suất,.. sẽ rất hữu ích trong đời sống con người nên nhóm em đã nghiên cứu và chọn đề tài: “Thiết kế một bộ đo nhiệt độ, độ ẩm và áp suất trên FPGA – Measuring Device”. Ý tưởng của bọn em là thiết kế một bo mạch đơn giản trên bo mạch trắng với các loại cảm biến như: DHT 11 (Cảm biến nhiệt độ, độ ẩm), BMP 280 (Cảm biến áp suất) để hiển thị lên LCD.

❖ Ý nghĩa khoa học và thực tiễn

* Ý nghĩa khoa học

Nghiên cứu và phát triển công nghệ: Dự án cung cấp cơ hội để nghiên cứu và phát triển công nghệ trong lĩnh vực của hệ thống nhúng sử dụng FPGA. Việc nắm vững và áp dụng các kiến thức về vi điều khiển, điện tử số, và vi xử lý logic trên FPGA là một phần quan trọng của sự tiến bộ trong ngành công nghiệp và nghiên cứu. Nghiên cứu trong lĩnh vực đo lường và cảm biến: Dự án đề cập đến việc sử dụng và tích hợp các cảm biến nhiệt độ, độ ẩm và áp suất khí quyển. Việc nghiên cứu về các công nghệ đo lường và cảm biến có thể dẫn đến sự tiến bộ trong việc phát triển các thiết bị đo lường chính xác và hiệu quả hơn. Ứng dụng của công nghệ nhúng trong thực tế: Dự án đề xuất sử dụng FPGA như một nền tảng để triển khai hệ thống đo đa năng. Việc áp dụng công nghệ nhúng vào các ứng dụng thực tế như đo lường

* Ý nghĩa thực tiễn

Ứng dụng trong các lĩnh vực công nghiệp và môi trường: Hệ thống đo đa năng có thể được áp dụng rộng rãi trong các lĩnh vực như công nghiệp, nông nghiệp, quản lý môi trường, và tự động hóa. Việc đo lường chính xác các thông số môi trường như nhiệt độ, độ ẩm và áp suất khí quyển là cực kỳ quan trọng để đảm bảo chất lượng và an toàn trong các quy trình sản xuất và quản lý môi trường. Sử dụng trong các ứng dụng cá nhân và gia đình: Hệ thống có thể được sử dụng trong các ứng dụng cá nhân và gia đình, chẳng hạn như thiết bị đo môi trường thông minh, hệ thống điều khiển tự động trong nhà, hoặc các ứng dụng liên quan đến việc giám

sát môi trường sống. Hỗ trợ trong quản lý và dự báo thời tiết: Dữ liệu từ hệ thống đo có thể được sử dụng để hỗ trợ trong việc quản lý và dự báo thời tiết. Thông tin về nhiệt độ, độ ẩm và áp suất khí quyển có thể giúp cải thiện độ chính xác của các dự báo thời tiết và cung cấp thông tin hữu ích cho các ứng dụng như nông nghiệp và điều hành hàng không.

ĐỐI TƯỢNG NGHIÊN CỨU

Trong bài tiểu luận này, đối tượng nghiên cứu chính bao gồm:

- *Phần mềm lập trình Quartus II*: Công cụ hỗ trợ thiết kế và mô phỏng các hệ thống số trên FPGA.
- *KIT thực hành Kit De0 Nano*: Phần cứng dùng để triển khai và kiểm tra hệ thống đèn giao thông được thiết kế.
- *Ngôn ngữ lập trình Verilog*: Ngôn ngữ mô tả phần cứng được sử dụng để thiết kế hệ thống.

MỤC TIÊU

Mục tiêu chính của bài tiểu luận này bao gồm:

- Hiểu và phân biệt được các mô hình lập trình Verilog: *Structural Verilog*, *Behavioral Verilog* và *Dataflow Verilog*.
- Nắm vững khái niệm và cách thức hoạt động của *Máy trạng thái hữu hạn (FSM)*.
- Rèn luyện tư duy lập trình phần cứng thông qua phương pháp “*think and code hardware*”.

NỘI DUNG NGHIÊN CỨU

Đề tài “**Thiết kế một bộ đo nhiệt độ, độ ẩm và áp suất trên FPGA – Measuring Device**” sẽ tập trung vào các nội dung chính sau:

1. Tìm hiểu các mô hình Verilog:

Structural Verilog: Mô tả cấu trúc phần cứng thông qua các module và kết nối giữa chúng.

Behavioral Verilog: Mô tả hành vi của hệ thống thông qua các quy trình và điều kiện.

Dataflow Verilog: Mô tả luồng dữ liệu và cách thức xử lý tín hiệu.

2. Nghiên cứu hoạt động của hệ thống đèn giao thông thực tế:

Phân tích các pha đèn, thời gian chuyển đổi giữa các pha, và cách thức điều khiển tại các ngã tư.

Tìm hiểu các yêu cầu về an toàn và hiệu quả trong thiết kế hệ thống đèn giao thông.

3. Thiết kế giao diện hiển thị và điều khiển:

Xây dựng giao diện hiển thị trực quan trên KIT DeO Nano

Thiết kế các nút điều khiển để thay đổi trạng thái đèn theo yêu cầu.

4. Lập trình ứng dụng FPGA bằng ngôn ngữ Verilog:

Viết code để mô tả hoạt động của hệ thống đèn giao thông.

Tối ưu hóa code để đảm bảo hiệu suất và độ tin cậy.

5. Kiểm tra và chỉnh sửa lỗi:

Sử dụng các công cụ mô phỏng để kiểm tra tính chính xác của

Phát hiện và sửa lỗi trong quá trình thiết kế và triển khai.

6. Đánh giá kết quả và viết báo cáo:

Đánh giá hiệu quả của hệ thống thông qua các tiêu chí như độ chính xác, thời gian phản hồi và khả năng mở rộng.

Tổng hợp kết quả và viết báo cáo chi tiết về quá trình thực hiện đề tài.

PHƯƠNG PHÁP NGHIÊN CỨU

Do đây là một đồ án sản phẩm, chúng tôi đã áp dụng phương pháp nghiên cứu kết hợp giữa lý thuyết và thực nghiệm. Cụ thể, quá trình nghiên cứu được thực hiện theo các bước sau:

1. Nghiên cứu lý thuyết:

Tìm hiểu các khái niệm cơ bản về lập trình FPGA, ngôn ngữ Verilog, và các mô hình lập trình như Structural, Behavioral, và Dataflow.

2. Thiết kế và lập trình:

-Sử dụng phần mềm Quartus II 13.1 để thiết kế và mô phỏng hệ thống đèn giao thông.

-Viết code bằng ngôn ngữ Verilog để mô tả hoạt động của hệ thống.

3. Thực nghiệm và kiểm tra:

- Tích hợp các module Verilog vào hệ thống chính trên FPGA.

- Kiểm tra chức năng của hệ thống và debug nếu cần.

4. Hoàn thiện sản phẩm:

- Tối ưu hóa code và giao diện hiển thị.

- Đánh giá kết quả và viết báo cáo tổng hợp quá trình thực hiện.

GIỚI HẠN

Đề tài “**Thiết kế một bộ đo nhiệt độ, độ ẩm và áp suất trên FPGA – Measuring Device**” có những giới hạn sau:

Phần mềm sử dụng: Quartus II 13.1 để lập trình ứng dụng FPGA.

Phần cứng sử dụng: KIT thực hành Kit De0 Nano để triển khai và kiểm tra hệ thống.

Phạm vi nghiên cứu: Tập trung vào thiết kế và mô phỏng hệ thống đo nhiệt độ độ ẩm và áp suất đơn giản, chưa mở rộng sang các hệ thống phức tạp hơn hoặc tích hợp với các hệ thống thông minh khác.

Hiện nay, việc lập trình ứng dụng FPGA đang được ứng dụng rộng rãi trong nhiều lĩnh vực tại Việt Nam, từ công nghiệp đến nghiên cứu khoa học. Trên cơ sở lý thuyết được học trong môn Lập trình ứng dụng FPGA và trong khuôn khổ của bài tiểu luận, chúng tôi đã thực hiện đề tài “**Thiết kế một bộ đo nhiệt độ, độ ẩm và áp suất trên FPGA – Measuring Device**” dưới sự hướng dẫn của TS. Lê Văn Thuận. Mục tiêu chính của chúng tôi là thiết kế một mô hình mô phỏng hệ thống đèn giao thông có hoạt động tương tự như hệ thống thực tế, từ đó củng cố kiến thức và kỹ năng lập trình phần cứng.

Tuy nhiên, do kiến thức và kinh nghiệm còn hạn chế, bài tiểu luận của chúng tôi không tránh khỏi những thiếu sót. Mặc dù đã thiết kế được hệ thống đèn giao thông cơ bản, sản phẩm vẫn mang tính lý thuyết nhiều và chưa được tối ưu hóa hoàn toàn. Chúng tôi rất mong nhận được sự góp ý, đánh giá và hướng dẫn từ các thầy cô để đề tài có thể được hoàn thiện hơn trong tương lai.

Cuối cùng, chúng tôi xin chân thành cảm ơn **TS. Lê Văn Thuận** đã tận tình hướng dẫn và hỗ trợ chúng tôi trong suốt quá trình thực hiện bài tiểu luận này.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1. Ngôn ngữ lập trình phần cứng Verilog HDL

Ngôn ngữ lập trình phần cứng (Hardware Description Language -Verilog HDL) là một công cụ quan trọng trong thiết kế và phát triển các hệ thống điện tử số. HDL cho phép mô tả cấu trúc và hành vi của các mạch điện tử, cũng như các tương tác giữa các thành phần trong mạch. Khác với các ngôn ngữ lập trình phần mềm thông thường, Verilog HDL tập trung vào việc mô tả cách thức hoạt động của phần cứng, từ đó tạo ra các mô hình logic có thể được triển khai trên các thiết bị phần cứng như FPGA (Field-Programmable Gate Array) hoặc ASIC (Application-Specific Integrated Circuit).

Các ngôn ngữ HDL phổ biến

Hai ngôn ngữ HDL được sử dụng rộng rãi nhất hiện nay là:

VHDL (Very High-Speed Integrated Circuit Hardware Description

Language): Một ngôn ngữ mạnh mẽ với cú pháp chặt chẽ, thường được sử dụng trong các dự án yêu cầu độ tin cậy cao, như trong lĩnh vực hàng không vũ trụ hoặc quốc phòng.

Verilog: Một ngôn ngữ có cú pháp đơn giản và dễ học hơn so với VHDL, thường được ưa chuộng trong các ứng dụng thiết kế FPGA và ASIC.

Cả hai ngôn ngữ đều cho phép mô tả các mạch điện tử ở nhiều mức độ trừu tượng khác nhau, từ mức hệ thống đến mức cổng logic.

Ngôn ngữ lập trình phần cứng HDL, đặc biệt là VHDL và Verilog, đóng vai trò quan trọng trong việc thiết kế và phát triển các hệ thống điện tử số hiện đại. Với khả năng mô tả, mô phỏng và kiểm tra các mạch điện tử một cách hiệu quả, HDL giúp rút ngắn thời gian phát triển, giảm thiểu rủi ro và nâng cao hiệu suất của các dự án phần cứng. Đây là công cụ không thể thiếu đối với các kỹ sư điện tử và những người làm việc trong lĩnh vực thiết kế hệ thống số.

VHDL	Verilog
<pre> library IEEE; use IEEE.STD_Logic_1164, all; entity LATCH_IF_ELSEIF is port (En1, En2, En3, A1, A2, A3: in std_logic; Y: out std_logic); end entity LATCH_IF_ELSEIF; architecture RTL of LATCH_IF_ELSEIF is begin process (En1, En2, En3, A1, A2, A3) begin if (En1 = '1') then Y <= A1; elseif (En2 = '1') then Y <= A2; elseif (En3 = '1') then Y <= A3; end if; end process; end architecture RTL; </pre>	<pre> module LATCH_IF_ELSEIF (En1, En2, En3, A1, A2, A3, Y); input En1, En2, En3, A1, A2, A3; output Y; reg Y; always @(En1 or En2 or En3 or A1 or A2 or A3) if (En1 == 1) Y = A1; else if (En2 == 1) Y = A2; else if (En3 == 1) Y = A3; end module </pre>

Hình 1: Ví dụ về HDL

2. Mạch tích hợp logic có thể lập trình được (FPGA)

FPGA (Field-Programmable Gate Array) là một loại chip điện tử có khả năng lập trình lại sau khi đã được sản xuất. Đây là một công nghệ phần cứng linh hoạt, được sử dụng rộng rãi trong các ứng dụng liên quan đến vi mạch và hệ thống phần cứng số. FPGA cho phép người dùng thiết kế và triển khai các mạch logic tùy chỉnh, giúp tối ưu hóa hiệu suất và tính linh hoạt trong các hệ thống điện tử.

Cấu trúc của FPGA

FPGA bao gồm các thành phần chính sau:

1. Lưới logic (*Logic cells*):

Là các khối logic cơ bản (thường là các Look-Up Tables - LUTs) có thể được cấu hình để thực hiện các chức năng logic khác nhau. Các khối logic này có thể kết hợp với nhau để tạo ra các mạch phức tạp hơn.

2. Các yếu tố kết nối (*Interconnect*):

Là một mạng lưới các đường dẫn có thể cấu hình để kết nối các khối logic với nhau.

Điều này cho phép người dùng tạo ra các kết nối tùy ý giữa các thành phần bên trong FPGA.

3. Bộ nhớ khối (*Block RAM*):

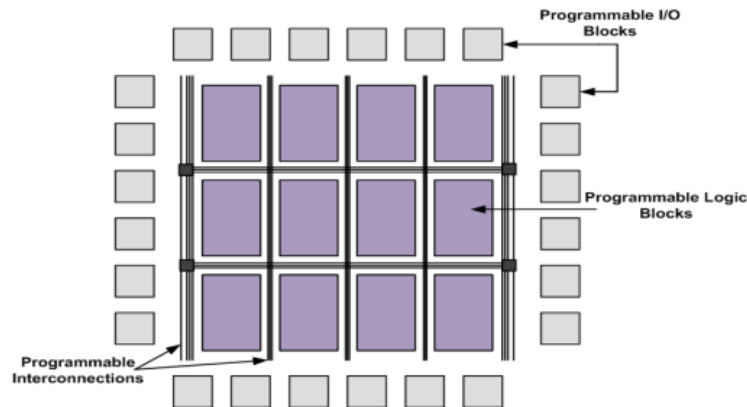
FPGA thường tích hợp các khối bộ nhớ để lưu trữ dữ liệu tạm thời hoặc thực hiện các chức năng xử lý tín hiệu.

4. Các khối xử lý tín hiệu số (*DSP Blocks*):

Các khối DSP được tích hợp sẵn để thực hiện các phép toán số học phức tạp, phục vụ cho các ứng dụng xử lý tín hiệu.

5. Các giao diện I/O (*Input/Output Blocks*):

Cho phép FPGA giao tiếp với các thiết bị bên ngoài thông qua các chuẩn giao tiếp khác nhau.



Hình 2 :Cấu trúc FPGA

Một FPGA (Field-Programmable Gate Array) bao gồm ba thành phần chính: khối CLB (Configurable Logic Block), khối kết nối (interconnect), và khối I/O (Input/Output).

1. Khối CLB (Configurable Logic Block): Đây là thành phần chính thực hiện các chức năng logic trong FPGA. Mỗi CLB bao gồm một hoặc nhiều mạch tổ hợp (LUT - Look-Up Table) và các flip-flop hoặc chốt (latch). LUT là một bảng tra cứu dùng để thực hiện các phép logic đơn giản, và flip-flop/chốt được sử dụng để lưu trữ dữ liệu. Trong một FPGA, có nhiều CLB được kết hợp lại để tạo thành mạch logic phức tạp.

2. Khối kết nối (Interconnect): Thành phần này có nhiệm vụ liên kết các CLB và các cổng I/O lại với nhau để tạo thành một mạch logic hoàn chỉnh. Mạng kết nối trong FPGA được xây dựng từ các đường kết nối ngang và đứng, và các khối chuyển mạch lập trình được sử dụng để kết nối các đường này. Khối kết nối cho phép dữ liệu di chuyển giữa các CLB và các cổng I/O theo đúng cấu trúc mạch logic được lập trình.

3. Khối I/O (Input/Output): Đây là thành phần dùng để giao tiếp giữa FPGA và các thiết bị ngoại vi hoặc các thành phần bên ngoài. Khối I/O bao gồm các chân ngõ vào (input) và ngõ ra (output), được sử dụng để truyền dữ liệu vào và ra khỏi FPGA. Các chân I/O có thể được cấu hình là đầu vào, đầu ra hoặc cả hai. Ngoài

ra, có các chân I/O được mã hóa cứng với các chức năng cụ thể như nguồn điện, cấu hình FPGA và tín hiệu clock.

Các thành phần này cùng hoạt động để tạo ra một mạch logic tùy chỉnh trên FPGA. Khối CLB thực hiện các phép logic và lưu trữ dữ liệu, khối kết nối liên kết các CLB và cổng I/O lại với nhau, và khối I/O cho phép giao tiếp với các thiết bị bên ngoài. Sự kết hợp của ba thành phần này cho phép người dùng tạo ra các mạch logic linh hoạt và tùy chỉnh trên FPGA

Nguyên lý hoạt động của FPGA

FPGA hoạt động dựa trên việc lập trình cấu hình các khối logic và kết nối giữa chúng. Người dùng sử dụng các ngôn ngữ mô tả phần cứng (HDL) như VHDL hoặc Verilog để mô tả chức năng và kết nối của mạch logic. Sau đó, các công cụ tổng hợp (synthesis tools) sẽ chuyển đổi mã HDL thành cấu hình bitstream, được tải lên FPGA để cấu hình lại chip theo thiết kế.

Ưu điểm của FPGA

Tính linh hoạt cao.

Khả năng xử lý song song.

Tiết kiệm chi phí và năng lượng.

Thời gian phát triển ngắn.

Ứng dụng của FPGA

FPGA được sử dụng rộng rãi trong nhiều lĩnh vực, bao gồm:

Viễn thông.

Điện tử tiêu dùng.

Ô tô tự động.

Thiết bị y tế.

Trí tuệ nhân tạo và học máy.

Nghiên cứu và phát triển.

FPGA là một công nghệ phần cứng linh hoạt và mạnh mẽ, đóng vai trò quan trọng trong việc thiết kế và triển khai các hệ thống số phức tạp. Với khả năng lập trình lại, xử lý song song và tiết kiệm chi phí, FPGA đã trở thành công cụ không thể thiếu trong nhiều lĩnh vực công nghệ cao. Sự phát triển của FPGA tiếp tục mở ra những cơ hội mới trong việc xây dựng các hệ thống phần cứng hiệu suất cao và linh hoạt.



Hình 3: Ví dụ về FPGA

3. Phần mềm lập trình Quartus II

Quartus II 13.1 là phiên bản thứ 13 của phần mềm thiết kế hệ thống số *Quartus Prime*, được phát triển bởi Intel (trước đây là Altera). Đây là một môi trường phát triển tích hợp (IDE) mạnh mẽ, được sử dụng rộng rãi trong việc thiết kế và triển khai phần cứng số trên các nền tảng FPGA (Field-Programmable Gate Array) và CPLD (Complex Programmable Logic Device).

Quartus II 13.1 ra mắt vào năm 2013 và mang đến nhiều công cụ và tính năng tiên tiến, hỗ trợ toàn diện quá trình thiết kế phần cứng. Dưới đây là những điểm nổi bật của phần mềm này:

Các tính năng chính của Quartus II

Môi trường thiết kế tích hợp (IDE):

Quartus II 13.1 cung cấp một giao diện đồ họa thân thiện, dễ sử dụng, cho phép người dùng thiết kế và mô phỏng các mạch điện tử trên FPGA.

Nó tích hợp các công cụ thiết kế, trình biên dịch và trình tổng hợp để tạo và xử lý các module logic, cũng như kết nối giữa chúng.

Hỗ trợ ngôn ngữ lập trình phần cứng:

Quartus II 13.1 hỗ trợ các ngôn ngữ lập trình phần cứng phổ biến như VHDL và Verilog.

Người dùng có thể sử dụng các ngôn ngữ này để mô tả các mạch logic, thiết kế các chức năng và triển khai chúng trên FPGA.

Công cụ mô phỏng và xác minh:

Quartus II 13.1 cung cấp các công cụ mô phỏng mạnh mẽ như *ModelSim* để kiểm tra tính đúng đắn và hiệu suất của mạch điện tử.

Người dùng có thể mô phỏng hoạt động của mạch trước khi triển khai trên phần cứng thực tế, giúp phát hiện và sửa lỗi kịp thời.

Bộ công cụ tổ chức và tối ưu hóa:

Phần mềm cung cấp các công cụ để tổ chức mã nguồn, tối ưu hóa thiết kế và giảm thiểu kích thước của mạch điện tử.

Các tính năng như Logic Optimization và Place-and-Route giúp cải thiện hiệu suất và độ tin cậy của thiết kế.

Hỗ trợ đa nền tảng FPGA:

Quartus II 13.1 hỗ trợ nhiều dòng sản phẩm FPGA của Intel/Altera, bao gồm các dòng Cyclone, Arria, và Stratix.

Điều này cho phép người dùng lựa chọn nền tảng phù hợp với yêu cầu và ứng dụng cụ thể của họ.

Tích hợp thư viện IP (*Intellectual Property*):

Quartus II 13.1 cung cấp các thư viện IP có sẵn, bao gồm các module chức năng như bộ nhớ, bộ xử lý tín hiệu số (DSP), và giao diện truyền thông.

Người dùng có thể tích hợp các IP này vào thiết kế của mình để tiết kiệm thời gian và công sức.

Hỗ trợ thiết kế hệ thống nhúng:

Quartus II 13.1 tích hợp công cụ Qsys để thiết kế các hệ thống nhúng trên FPGA, bao gồm việc kết nối các bộ xử lý (như Nios II) với các ngoại vi và bộ nhớ.

Lợi ích của Quartus II

Tăng tốc độ phát triển: Với các công cụ mạnh mẽ và thư viện IP phong phú, Quartus II 13.1 giúp rút ngắn thời gian thiết kế và triển khai hệ thống.

Độ chính xác cao: Các công cụ mô phỏng và xác minh giúp đảm bảo tính chính xác của thiết kế trước khi triển khai trên phần cứng thực tế.

Tối ưu hóa hiệu suất: Các tính năng tối ưu hóa giúp cải thiện hiệu suất và giảm tiêu thụ năng lượng của hệ thống.

Linh hoạt và dễ sử dụng: Giao diện thân thiện và hỗ trợ đa nền tảng giúp người dùng dễ dàng làm việc với phần mềm.

Ứng dụng của Quartus II

Quartus II 13.1 được sử dụng rộng rãi trong nhiều lĩnh vực, bao gồm:

Thiết kế vi mạch: Phát triển các hệ thống số phức tạp trên FPGA và CPLD.

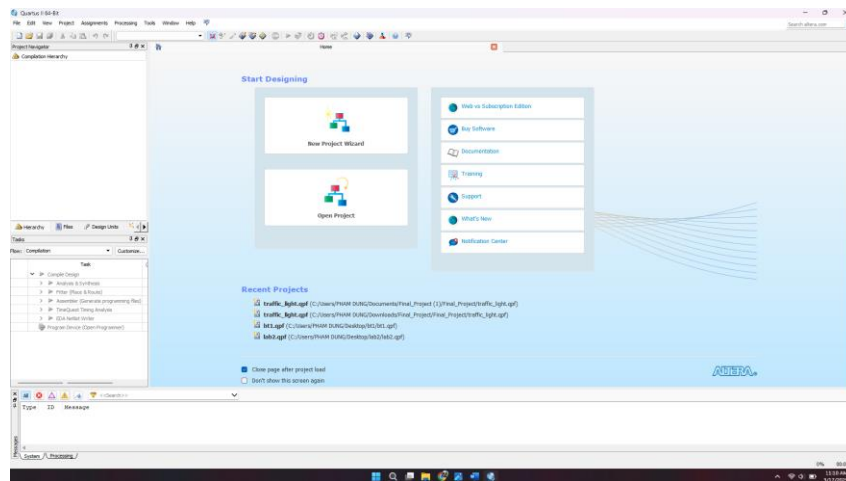
Hệ thống nhúng: Thiết kế các hệ thống điều khiển và xử lý tín hiệu thời gian thực.

Nghiên cứu và giáo dục: Hỗ trợ giảng dạy và nghiên cứu trong lĩnh vực thiết kế phần cứng.

Công nghiệp và viễn thông: Ứng dụng trong các hệ thống truyền thông, xử lý tín hiệu và điều khiển công nghiệp.

Quartus II 13.1 là một công cụ thiết kế phần cứng số mạnh mẽ và linh hoạt, được sử dụng rộng rãi trong việc phát triển các hệ thống FPGA và CPLD. Với các tính năng tiên tiến như mô phỏng, tối ưu hóa và hỗ trợ đa nền tảng, Quartus II

13.1 giúp người dùng thiết kế và triển khai các mạch điện tử phức tạp một cách hiệu quả và chính xác. Đây là một phần mềm không thể thiếu đối với các kỹ sư và nhà nghiên cứu trong lĩnh vực thiết kế phần cứng số.



Hình 4: Giao diện phần mềm lập trình Quartus II

4 Testbench

Testbench Verilog là một mô-đun Verilog được sử dụng để kiểm tra và xác nhận tính đúng đắn của mạch điện tử số học được mô phỏng bằng ngôn ngữ mô tả phần cứng Verilog. Một testbench Verilog có thể được coi như một chương trình kiểm tra để đảm bảo rằng mạch được thiết kế hoạt động đúng theo đặc tả.

Testbench có nhiệm vụ tạo ra tín hiệu vào, áp dụng chúng vào mạch được kiểm tra và sau đó kiểm tra kết quả đầu ra của mạch với kết quả mong đợi. Nó giúp xác minh tính đúng đắn của mạch trong các điều kiện khác nhau và các trường hợp kiểm tra.

Một testbench Verilog bao gồm các thành phần sau:

Khai báo: Đầu tiên, cần khai báo các mạch con hoặc module được kiểm tra trong testbench. Điều này thường được thực hiện bằng cách đưa ra tên và kiểu dữ liệu của các mạch con.

Khởi tạo: Sau khi khai báo các mạch con, ta có thể khởi tạo chúng bằng cách gán giá trị cho các tín hiệu vào mạch.

Điều khiển: Testbench cần điều khiển quá trình kiểm tra. Điều này có thể bao gồm việc áp dụng các tín hiệu vào mạch, đợi cho mạch hoạt động và sau đó so sánh kết quả đầu ra với kết quả mong đợi.

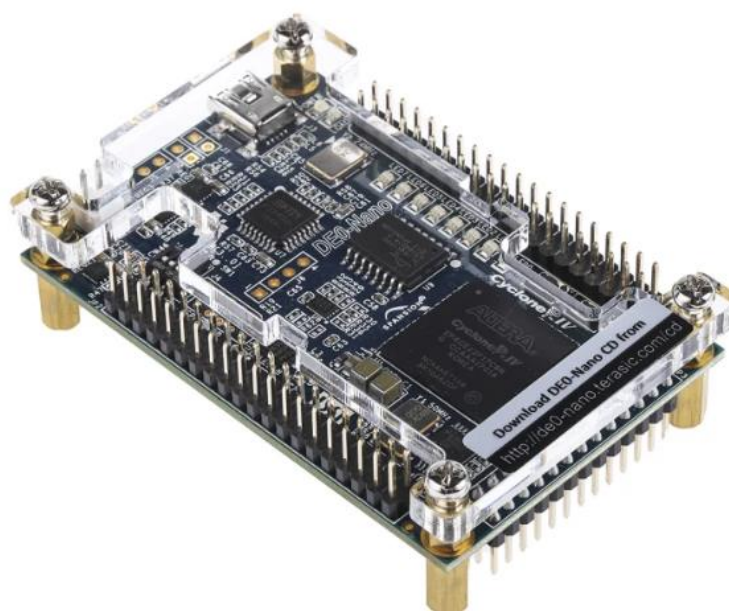
Kiểm tra: Testbench cần chứa các câu lệnh kiểm tra để so sánh kết quả đầu ra của mạch với kết quả mong đợi. Nếu kết quả không khớp, testbench sẽ báo cáo lỗi và hiển thị thông tin chi tiết về lỗi.

Hiển thị kết quả: Cuối cùng, testbench có thể cung cấp thông tin về kết quả kiểm tra, chẳng hạn như số lỗi tìm thấy, thời gian chạy, hoặc bất kỳ thông tin khác liên quan đến quá trình kiểm tra.

Testbench Verilog thường được sử dụng trong quá trình phát triển và kiểm tra các mạch số học phức tạp trước khi triển khai trên các thiết bị thực tế. Nó giúp đảm bảo tính đúng đắn và tin cậy của mạch điện tử trước khi sản xuất hàng loạt.

5 Kit De0 Nano

Kit DE0 Nano là một kit phát triển FPGA nhỏ gọn được sản xuất bởi Terasic, một công ty chuyên về thiết kế và phát triển các sản phẩm FPGA. Kit này được thiết kế với mục đích hỗ trợ việc học tập, nghiên cứu và phát triển các ứng dụng trên nền tảng FPGA



Hình 5 Kit DeO Nano

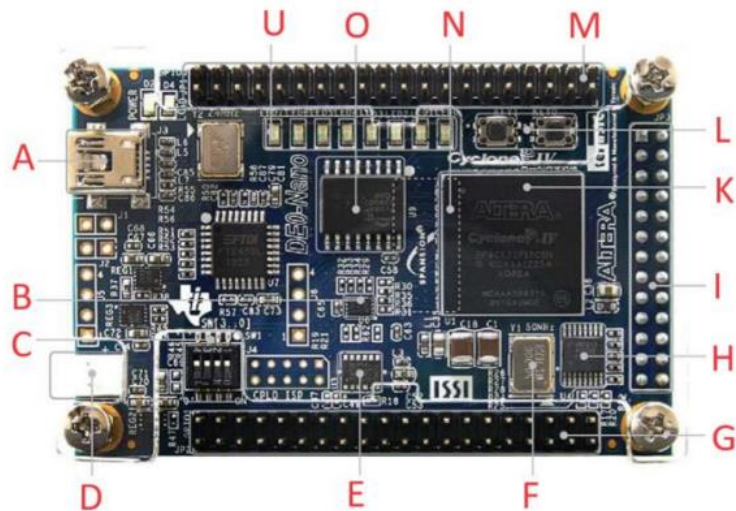
FPGA: Kit DE0 Nano sử dụng một chip FPGA Altera Cyclone IV EP4CE22F17C6N. FPGA là viết tắt của Field-Programmable Gate Array, đây là một loại vi mạch có khả năng được lập trình lại theo ý muốn. FPGA cho phép người dùng tạo ra các mạch logic tùy chỉnh và triển khai chúng trên kit.

Tài nguyên: Kit DE0 Nano cung cấp một loạt các tài nguyên để hỗ trợ việc phát triển trên FPGA. Điều này bao gồm các nút nhấn, công tắc, LED, hiển thị LCD, cổng GPIO, giao diện VGA, cổng âm thanh, cổng USB, và nhiều nút bấm khác. Các tài nguyên này cho phép người dùng tương tác với FPGA và xây dựng các ứng dụng thực tế.

Kết nối và mở rộng: Kit DE0 Nano cung cấp các cổng kết nối như cổng USB, cổng VGA, cổng âm thanh và cổng GPIO để kết nối với các thiết bị ngoại vi hoặc mở rộng chức năng của kit. Người dùng có thể kết nối kit với máy tính để lập trình và giao tiếp với FPGA.

Phần mềm hỗ trợ: Kit DE0 Nano được hỗ trợ bởi phần mềm Quartus Prime của Intel (trước đây là Altera). Phần mềm này cung cấp một giao diện đồ họa cho phép người dùng thiết kế, mô phỏng và triển khai các mạch logic trên FPGA. Kit DE0 Nano là một công cụ hữu ích để học tập và phát triển các ứng dụng trên nền tảng FPGA. Nó được sử dụng rộng rãi trong lĩnh vực giáo dục, nghiên cứu và phát triển các sản phẩm điện tử.

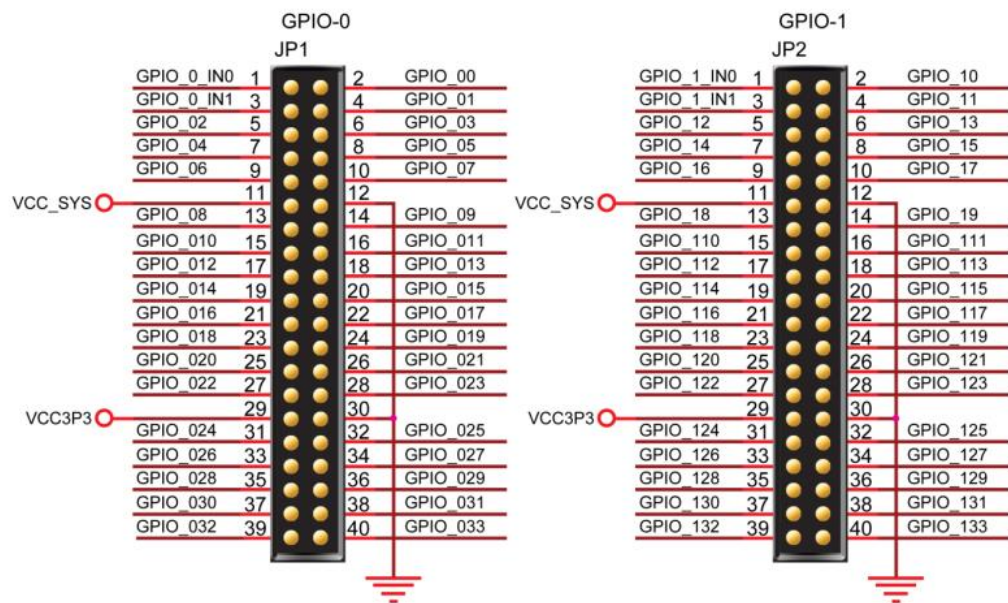
*Cấu trúc của kit De0 Nano



Hình 6 Cấu trúc của Kit DeO Nano

- A: Cổng USB mini type AB
- B: 2KB I2C EEPROM
- C: 4 cần gạt (dip switch)
- D: 2-pin External power header
- E: Cảm biến đo gia tốc
- F: Tần số xung clock 50 MHz
- G: 40 chân Pin GPIO – 1 (JP2)
- H: Chuyển đổi A/V
- I: 26 Pin header
- K: chip Altera Cyclone IV FPGA 21
- L: 2 công tắc button
- M: 40 chân GPIO – 0 (JP1)
- N: Dung lượng lưu trữ 32MB
- O: FPGA serial configuration device
- U: 8 đèn led màu xanh

*Cấu trúc PIN của GPIO



Hình 7 : Sơ đồ chân PIN của GPIO

Signal Name	FPGA Pin No.	Description	I/O Standard
GPIO_0_IN0	PIN_A8	GPIO Connection DATA	3.3V
GPIO_00	PIN_D3	GPIO Connection DATA	3.3V
GPIO_0_IN1	PIN_B8	GPIO Connection DATA	3.3V
GPIO_01	PIN_C3	GPIO Connection DATA	3.3V
GPIO_02	PIN_A2	GPIO Connection DATA	3.3V
GPIO_03	PIN_A3	GPIO Connection DATA	3.3V
GPIO_04	PIN_B3	GPIO Connection DATA	3.3V
GPIO_05	PIN_B4	GPIO Connection DATA	3.3V
GPIO_06	PIN_A4	GPIO Connection DATA	3.3V
GPIO_07	PIN_B5	GPIO Connection DATA	3.3V

GPIO_08	PIN_A5	GPIO Connection DATA	3.3V
GPIO_09	PIN_D5	GPIO Connection DATA	3.3V
GPIO_010	PIN_B6	GPIO Connection DATA	3.3V
GPIO_011	PIN_A6	GPIO Connection DATA	3.3V
GPIO_012	PIN_B7	GPIO Connection DATA	3.3V
GPIO_013	PIN_D6	GPIO Connection DATA	3.3V
GPIO_014	PIN_A7	GPIO Connection DATA	3.3V
GPIO_015	PIN_C6	GPIO Connection DATA	3.3V
GPIO_016	PIN_C8	GPIO Connection DATA	3.3V
GPIO_017	PIN_E7	GPIO Connection DATA	3.3V
GPIO_018	PIN_D8	GPIO Connection DATA	3.3V
GPIO_019	PIN_E8	GPIO Connection DATA	3.3V
GPIO_020	PIN_F8	GPIO Connection DATA	3.3V
GPIO_021	PIN_F9	GPIO Connection DATA	3.3V
GPIO_022	PIN_F9	GPIO Connection DATA	3.3V
GPIO_023	PIN_E9	GPIO Connection DATA	3.3V
GPIO_024	PIN_C9	GPIO Connection DATA	3.3V
GPIO_025	PIN_D9	GPIO Connection DATA	3.3V
GPIO_026	PIN_E11	GPIO Connection DATA	3.3V
GPIO_027	PIN_E10	GPIO Connection DATA	3.3V
GPIO_028	PIN_C11	GPIO Connection DATA	3.3V
GPIO_029	PIN_B11	GPIO Connection DATA	3.3V
GPIO_030	PIN_A12	GPIO Connection DATA	3.3V

GPIO_031	PIN_D11	GPIO Connection DATA	3.3V
GPIO_032	PIN_D12	GPIO Connection DATA	3.3V
GPIO_033	PIN_B12	GPIO Connection DATA	3.3V

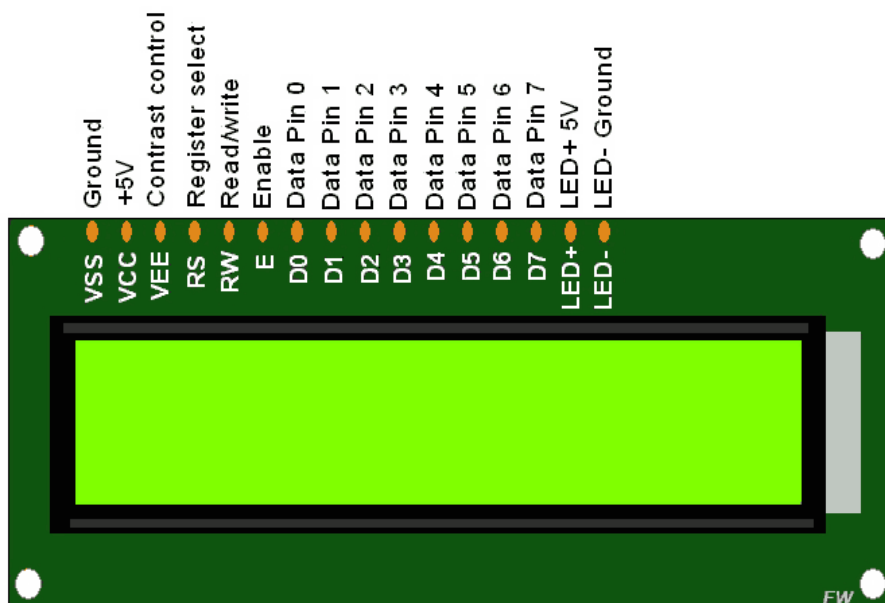
Bảng 1 : Bảng gán chân GPIO

6. LCD 16x2

thiết bị hiển thị LCD 16x2 (Liquid Crystal Display) được sử dụng trong rất nhiều các ứng dụng của VĐK. LCD 1602 có rất nhiều ưu điểm so với các dạng hiển thị khác như: khả năng hiển thị kí tự đa dạng (chữ, số, kí tự đồ họa); dễ dàng đưa vào mạch ứng dụng theo nhiều giao thức giao tiếp khác nhau, tiêu tốn rất ít tài nguyên hệ thống, giá thành rẻ,...

Thông số kĩ thuật của sản phẩm LCD 16x2:

- Điện áp MAX : 7V
- Điện áp MIN : - 0,3V
- Hoạt động ổn định : 2.7-5.5V
- Điện áp ra mức cao : > 2.4
- Điện áp ra mức thấp : <0.4V
- Dòng điện cấp nguồn : 350uA - 600uA
- Nhiệt độ hoạt động : - 30 - 75 độ C



Hình 8 : LCD 16x2

Chức năng của từng chân LCD 16x2:

- Chân số 1 - VSS : chân nối đất cho LCD được nối với GND của mạch điều khiển
- Chân số 2 - VDD : chân cấp nguồn cho LCD, được nối với VCC=5V của mạch điều khiển
- Chân số 3 - VE : điều chỉnh độ tương phản của LCD
- Chân số 4 - RS : chân chọn thanh ghi, được nối với logic "0" hoặc logic "1":
 - + Logic "0": Bus DB0 - DB7 sẽ nối với thanh ghi lệnh IR của LCD (ở chế độ "ghi" - write) hoặc nối với bộ đếm địa chỉ của LCD (ở chế độ "đọc" - read)
 - + Logic "1": Bus DB0 - DB7 sẽ nối với thanh ghi dữ liệu DR bên trong LCD
- Chân số 5 - R/W : chân chọn chế độ đọc/ghi (Read/Write), được nối với logic "0" để ghi hoặc nối với logic "1" đọc
- Chân số 6 - E : chân cho phép (Enable). Sau khi các tín hiệu được đặt lên bus DB0-DB7, các lệnh chỉ được chấp nhận khi có 1 xung cho phép của chân này như sau:
 - + Ở chế độ ghi: Dữ liệu ở bus sẽ được LCD chuyển vào thanh ghi bên trong khi phát hiện một xung (high-to-low transition) của tín hiệu chân E
 - + Ở chế độ đọc: Dữ liệu sẽ được LCD xuất ra DB0-DB7 khi phát hiện cạnh lên (low-to-high transition) ở chân E và được LCD giữ ở bus đến khi nào chân E xuống mức thấp
- Chân số 7 đến 14 - D0 đến D7: 8 đường của bus dữ liệu dùng để trao đổi thông tin với MPU. Có 2 chế độ sử dụng 8 đường bus này là: Chế độ 8 bit (dữ liệu được truyền trên cả 8 đường, với bit MSB là bit DB7) và Chế độ 4 bit (dữ liệu được truyền trên 4 đường từ DB4 tới DB7, bit MSB là DB7)
- Chân số 15 - A : nguồn dương cho đèn nền
- Chân số 16 - K : nguồn âm cho đèn nền

LỆNH	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0	Mô tả	clock
NOP	0	0	0	0	0	0	0	0	0	0	No operation	0
Clear display	0	0	0	0	0	0	0	0	0	1	Clear display & sets address counter to zero	165
Cursor home	0	0	0	0	0	0	0	0	1	0	Sets address counter to zero, returns shifted display to original position. DDRAM contents remain unchanged.	3
Entry mode set 0000 0110 = 0x06	0	0	0	0	0	0	0	1	I/D	S	Sets cursor move direction, and specifies automatic shift.	3
Display control	0	0	0	0	0	0	1	D	C	B	Turns display (D), cursor on/off (C) or cursor blinking (B).	3
Cursor/display shift	0	0	0	0	0	1	S/C	R/L	0	0	Move cursor and shift display. DDRAM contents remain unchanged.	3
Function set	0	0	0	0	1	DL	N	M	G	0	Sets interface data width (DL), number of display lines (N,M) and voltage generator control (G).	3
Set CGRAM addr	0	0	0	1	Character Generator RAM						Sets CGRAM address	3
Set DDRAM addr	0	0	1	Display data ram address							Sets DDRAM address	3
Buzy flag & Addr	0	0	BF	Address counter							Reads buzy flag & address counter	0
Read data	1	0	Read data								Reads data from CCGRAM or DDRAM	3
Read data	1	1	Write data								Write data to CCGRAM or DDRAM	3

Bảng 2 : Bảng lệnh điều khiển LCD
Các ký tự hiển thị lên LCD theo mã ACSII

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	\$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

Bảng 3: Bảng mã ASCII

Cảm Biến Nhiệt Độ Và Độ Ẩm DHT11

DHT11 là gì

DHT11 là một cảm biến kỹ thuật số giá rẻ để cảm nhận nhiệt độ và độ ẩm. Cảm biến này có thể dễ dàng giao tiếp với bất kỳ bộ vi điều khiển vi nào như Arduino, Raspberry Pi, ... để đo độ ẩm và nhiệt độ ngay lập tức.

DHT11 là một cảm biến độ ẩm tương đối. Để đo không khí xung quanh, cảm biến này sử dụng một điện trở nhiệt và một cảm biến độ ẩm điện dung.

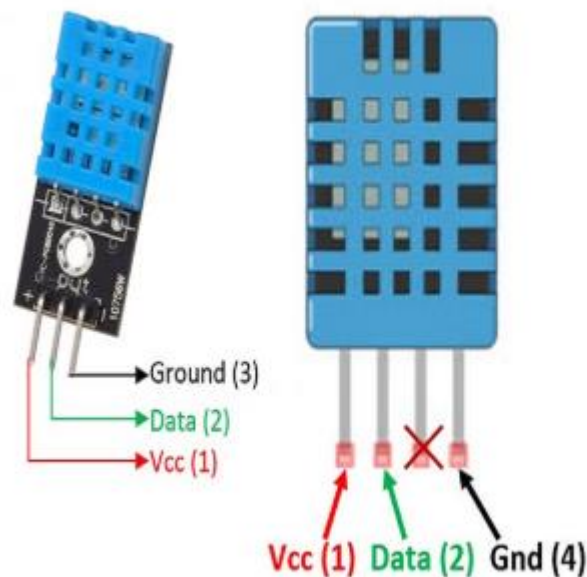
Cấu tạo cảm biến nhiệt độ độ ẩm DHT11

Cảm biến DHT11 bao gồm một phần tử cảm biến độ ẩm điện dung và một điện trở nhiệt để cảm nhận nhiệt độ. Tự điện cảm biến độ ẩm có hai điện cực với chất nền giữ ẩm làm chất điện môi giữa chúng. Thay đổi giá trị điện dung xảy ra với sự thay đổi của các mức độ ẩm. IC đo, xử lý các giá trị điện trở đã thay đổi này và chuyển chúng thành dạng kỹ thuật số.

Để đo nhiệt độ, cảm biến này sử dụng một nhiệt điện trở có hệ số nhiệt độ âm, làm giảm giá trị điện trở của nó khi nhiệt độ tăng. Để có được giá trị điện trở lớn hơn ngay cả đối với sự thay đổi nhỏ nhất của nhiệt độ, cảm biến này thường được làm bằng gốm bán dẫn hoặc polymer.

Sơ đồ chân DHT11

Số chân	Tên chân	Mô tả
1	Vcc	Nguồn 3.5V đến 5.5V
2	Data	Đầu ra cả nhiệt độ và độ ẩm thông qua dữ liệu nối tiếp
3	NC	Không có kết nối và do đó không sử dụng
4	Ground	Nối đất



Hình 9: Cảm biến nhiệt độ độ ẩm

Tính năng

- Cảm Biến Nhiệt Độ Và Độ Ẩm DHT11 là cảm biến rất thông dụng hiện nay vì chi phí rẻ và rất dễ lấy dữ liệu thông qua giao tiếp 1 wire (giao tiếp digital 1 dây truyền dữ liệu duy nhất). Bộ tiền xử lý tín hiệu tích hợp trong cảm biến giúp bạn có được dữ liệu chính xác mà không phải qua bất kỳ tính toán nào. So với cảm biến đời mới hơn là DHT22 thì DHT11 cho khoảng đo và độ chính xác kém hơn rất nhiều.

Thông số kỹ thuật DHT11

- Điện áp hoạt động: 3V - 5V DC
- Dòng điện tiêu thụ: 2.5mA
- Phạm vi cảm biến độ ẩm: 20% - 90% RH, sai số $\pm 5\%RH$
- Phạm vi cảm biến nhiệt độ: $0^{\circ}C \sim 50^{\circ}C$, sai số $\pm 2^{\circ}C$
- Tần số lấy mẫu tối đa: 1Hz (1 giây 1 lần)

- Kích thước: 23 * 12 * 5 mm

nguyên lý làm việc

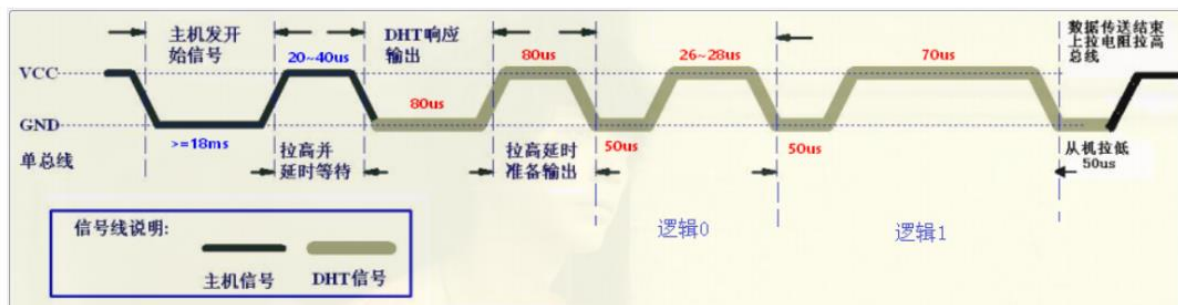
DHT11 sử dụng một bus duy nhất để giao tiếp, tức là đường dây kết nối chân DATA và vi điều khiển. Bus luôn ở **trạng thái Nhàn rỗi** với **Trạng thái giao tiếp** Giữa 2 trạng thái này.

Khi vi điều khiển không tương tác với DHT11, bus ở trạng thái nhàn rỗi và ở trạng thái mức cao dưới tác động của điện trở kéo lên.

Khi MCU và DHT11 đang giao tiếp, bus ở trạng thái giao tiếp, toàn bộ quá trình giao tiếp như sau:

- ① Vi điều khiển định cấu hình IO của bus ổ đĩa làm chế độ đầu ra. Sẵn sàng gửi dữ liệu đến DHT11.
- ② Vi điều khiển kéo bus xuống ít nhất 18ms để gửi tín hiệu bắt đầu. Sau đó kéo bus lên cao và trì hoãn 20~40us để biểu thị kết thúc tín hiệu bắt đầu.
- ③ Máy vi tính chip đơn định cấu hình IO của bus ổ đĩa làm chế độ đầu vào, sẵn sàng nhận dữ liệu do DHT11 trả về.
- ④ Khi DHT11 phát hiện tín hiệu khởi động được gửi bởi vi điều khiển, nó bắt đầu phản hồi và trả về dữ liệu cảm biến đã thu thập được. Đầu tiên, DHT11 kéo bus xuống mức thấp 80us như một xác nhận (ACK) cho vi điều khiển, sau đó kéo bus lên mức cao 80us, sẵn sàng trả về dữ liệu nhiệt độ và độ ẩm đã thu thập được. Dữ liệu nhiệt độ và độ ẩm được gửi theo định dạng khung cố định.

Một khung là 40 bit và logic thời gian truyền của mỗi bit là: mỗi bit được dẫn bởi mức thấp 50us (DHT11 kéo bus xuống mức thấp), sau đó DHT11 kéo bus lên mức cao. Nếu mức cao kéo dài trong 26~28us, nó biểu thị logic 0 và nếu kéo dài trong 70us, nó biểu thị logic 1.



Hình : Trạng thái của DHT11

Khi xem sơ đồ tín hiệu này, bạn có thể biết được chế độ giao tiếp giữa mô-đun và bộ điều khiển chính.

1. Chờ 1 giây sau khi bật nguồn để vượt qua trạng thái không ổn định.
2. IO là chế độ đầu ra tín hiệu, mức thấp kéo dài hơn 18ms. (khuyến nghị 20ms)
3. Mức IO được kéo lên cao trong khoảng 20~40us. (khuyến nghị 30us)
4. IO là chế độ đầu vào tín hiệu và nhập thời gian phản hồi DHT. (Tức là mô-đun nhận thông tin của bộ điều khiển chính và trả về tín hiệu gửi dữ liệu trước)
5. Độ trễ mức thấp là 80us và mức được kéo lên 80us để hoàn tất phản hồi DHT.
6. Nhập logic dữ liệu 0/1 để gửi, kéo dài 40 bit.
7. Logic 0, mức độ được kéo xuống trong 50us và mức độ được kéo lên trong 25us.
8. Logic 1, mức độ được kéo xuống trong 50us và mức độ được kéo lên trong 80us.
9. Sau khi dữ liệu được gửi đi, bus được kéo lên cao. (Ở đây có thể trì hoãn ít nhất 1 giây, sau đó thực hiện thu thập dữ liệu tiếp theo)

7 Các loại mô hình Verilog

Verilog là một ngôn ngữ mô tả phần cứng (HDL) mạnh mẽ, được sử dụng rộng rãi trong thiết kế các hệ thống số. Verilog hỗ trợ nhiều mô hình mô tả khác nhau, bao gồm Structural, Behavioral, và Dataflow. Mỗi mô hình có những đặc điểm và ứng dụng riêng, phù hợp với các giai đoạn khác nhau trong quá trình thiết kế phần cứng

❖ *Structural Verilog*

Structural Verilog là một phương pháp mô tả mạch điện tử dựa trên cấu trúc vật lý của hệ thống. Trong mô hình này, mạch điện tử được xem như một tập hợp các thành phần (module) độc lập, được kết nối với nhau thông qua các tín hiệu (wires hoặc bus). Mỗi thành phần có thể là một module con, một cổng logic, hoặc một khối chức năng được định nghĩa trước.

Đặc điểm của Structural Verilog

Mô tả theo cấu trúc:

Mạch điện tử được mô tả bằng cách kết hợp các module hoặc thành phần sẵn có. Các module này được kết nối với nhau thông qua các dây dẫn (wires) hoặc bus.

Tái sử dụng module:

Các module đã được thiết kế trước đó có thể được tái sử dụng trong các thiết kế mới, giúp tiết kiệm thời gian và công sức.

Gắn gũi với phần cứng thực tế:

Structural Verilog mô tả mạch điện tử theo cách gần với cấu trúc vật lý thực tế, giúp người thiết kế dễ dàng hình dung và kiểm tra tính đúng đắn của mạch.

Hỗ trợ thiết kế phân cấp:

Cho phép chia nhỏ thiết kế thành các module con, giúp quản lý và phát triển hệ thống phức tạp một cách dễ dàng.

Ưu điểm của Structural Verilog

Dễ hiểu và dễ kiểm tra: Mô tả mạch theo cấu trúc giúp người thiết kế dễ dàng kiểm tra tính đúng đắn của mạch.

Tái sử dụng mã nguồn: Các module đã được thiết kế có thể được sử dụng lại trong các dự án khác.

Phù hợp với thiết kế phần cứng: Structural Verilog gần gũi với cấu trúc vật lý của mạch, giúp tối ưu hóa hiệu suất và tài nguyên phần cứng.

Nhược điểm của Structural Verilog

Khó mô tả các hệ thống phức tạp: Đối với các hệ thống lớn và phức tạp, việc mô tả bằng Structural Verilog có thể trở nên cồng kềnh và khó quản lý.

Yêu cầu kiến thức về phần cứng: Người thiết kế cần hiểu rõ về cấu trúc vật lý của mạch để sử dụng hiệu quả.

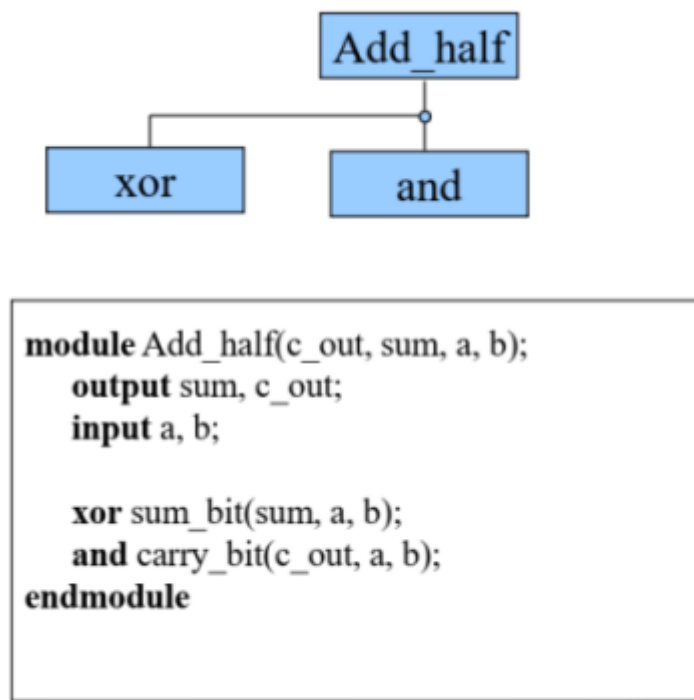
Ứng dụng của Structural Verilog

Thiết kế các mạch logic cơ bản: Như bộ cộng, bộ so sánh, bộ đếm, v.v.

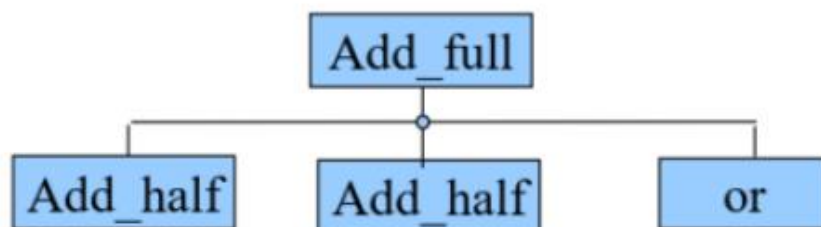
Xây dựng các hệ thống phân cấp: Khi cần chia nhỏ hệ thống thành các module con để quản lý dễ dàng.

Tái sử dụng các thiết kế đã có: Khi cần tích hợp các module đã được thiết kế và kiểm tra trước đó.

Structural Verilog là một phương pháp mô tả mạch điện tử dựa trên cấu trúc vật lý, giúp người thiết kế dễ dàng hình dung và kiểm tra tính đúng đắn của mạch. Với khả năng tái sử dụng mã nguồn và hỗ trợ thiết kế phân cấp, Structural Verilog là một công cụ mạnh mẽ trong thiết kế các hệ thống số phức tạp. Tuy nhiên, đối với các hệ thống quá lớn, việc kết hợp với các mô hình khác như Behavioral hoặc Dataflow có thể mang lại hiệu quả cao hơn.



Hình 10: Bộ cộng bán phần



```

module Add_full(c_out, sum, a, b, c_in) ;
  output sum, c_out;
  input a, b, c_in;
  wire w1, w2, w3;

  Add_half AH1(.sum(w1), .c_out(w2), .a(a), .b(b));
  Add_half AH2(.sum(sum), .c_out(w3), .a(c_in), .b(w1));
  or carry_bit(c_out, w2, w3);
endmodule

```

Hình 11: Bộ cộng toàn phần

❖ Behavioral Verilog

Behavioral Verilog là một phương pháp mô tả mạch điện tử dựa trên hành vi và luồng hoạt động của hệ thống. Khác với *Structural Verilog*, *Behavioral Verilog* không tập trung vào cấu trúc vật lý của mạch mà thay vào đó mô tả cách thức hoạt động của mạch thông qua các khối mã nguồn (blocks of code).

Đặc điểm của Behavioral Verilog

Mô tả hành vi:

Behavioral Verilog tập trung vào việc mô tả chức năng và luồng hoạt động của mạch.

Các câu lệnh như if, case, for, và while được sử dụng để mô tả logic điều khiển. Không phụ thuộc vào cấu trúc phần cứng:

Behavioral Verilog không yêu cầu người thiết kế phải hiểu rõ về cấu trúc vật lý của mạch.

Thay vào đó, nó tập trung vào việc mô tả cách mạch hoạt động dựa trên các tín hiệu đầu vào và đầu ra.

Dễ đọc và dễ hiểu:

Với cách tiếp cận dựa trên hành vi, mã nguồn *Behavioral Verilog* thường dễ đọc và dễ hiểu hơn so với *Structural Verilog*.

Phù hợp cho mô phỏng và kiểm tra:

Behavioral Verilog thường được sử dụng trong giai đoạn mô phỏng và kiểm tra tính đúng đắn của thiết kế trước khi chuyển sang giai đoạn triển khai phần cứng.

Ưu điểm của Behavioral Verilog

Dễ mô tả các hệ thống phức tạp: *Behavioral Verilog* phù hợp để mô tả các hệ thống có logic điều khiển phức tạp.

Tập trung vào chức năng: Người thiết kế có thể tập trung vào việc mô tả chức năng của mạch mà không cần quan tâm đến cấu trúc phần cứng.

Dễ dàng mô phỏng và kiểm tra: *Behavioral Verilog* thường được sử dụng trong giai đoạn mô phỏng để kiểm tra tính đúng đắn của thiết kế.

Nhược điểm của Behavioral Verilog

Khó tối ưu hóa phần cứng: Vì không tập trung vào cấu trúc vật lý, *Behavioral Verilog* có thể không tối ưu về tài nguyên phần cứng.

Khó chuyển đổi sang phần cứng thực tế: Cần phải chuyển đổi mã *Behavioral Verilog* sang *Structural* hoặc *Dataflow* để triển khai trên phần cứng.

❖ *Dataflow Verilog*

Dataflow Verilog là một phương pháp mô tả mạch điện tử dựa trên luồng dữ liệu và các phép toán logic. Trong mô hình này, mạch điện tử được mô tả thông qua các quan hệ giữa các tín hiệu và các phép toán trên chúng.

Đặc điểm của Dataflow Verilog

Mô tả luồng dữ liệu:

Dataflow Verilog tập trung vào việc mô tả cách dữ liệu di chuyển và được xử lý trong mạch.

Các phép toán logic như AND, OR, NOT được sử dụng để mô tả quan hệ giữa các tín hiệu.

Ngắn gọn và trực quan:

Mã nguồn Dataflow Verilog thường ngắn gọn và dễ hiểu, phù hợp để mô tả các mạch logic đơn giản.

Không phù hợp cho các mạch có tính chất thời gian:

Dataflow Verilog không phù hợp để mô tả các mạch đồng bộ hoặc các mạch có yêu cầu về thời gian.

Ưu điểm của Dataflow Verilog:

Ngắn gọn và dễ hiểu: Mã nguồn Dataflow Verilog thường ngắn gọn và dễ đọc.

Phù hợp cho các mạch logic đơn giản: Dataflow Verilog thích hợp để mô tả các mạch logic cơ bản.

Nhược điểm của Dataflow Verilog

Không phù hợp cho các mạch phức tạp: Dataflow Verilog không phù hợp để mô tả các mạch có logic điều khiển phức tạp hoặc các mạch đồng bộ.

Khó mô tả hành vi thời gian thực: Dataflow Verilog không hỗ trợ mô tả các yêu cầu về thời gian hoặc các sự kiện đồng bộ.

8 Các loại máy trạng thái FSM

Máy trạng thái Mealy

Máy trạng thái Mealy là một loại máy trạng thái hữu hạn (Finite State Machine - FSM) trong đó tín hiệu đầu ra phụ thuộc vào cả trạng thái hiện tại và tín hiệu đầu vào.

Đặc điểm của máy trạng thái Mealy:

Tín hiệu đầu ra: Phụ thuộc vào cả trạng thái hiện tại và tín hiệu đầu vào.

Phản ứng nhanh với tín hiệu đầu vào: Tín hiệu đầu ra có thể thay đổi ngay lập tức khi có sự thay đổi trên tín hiệu đầu vào.

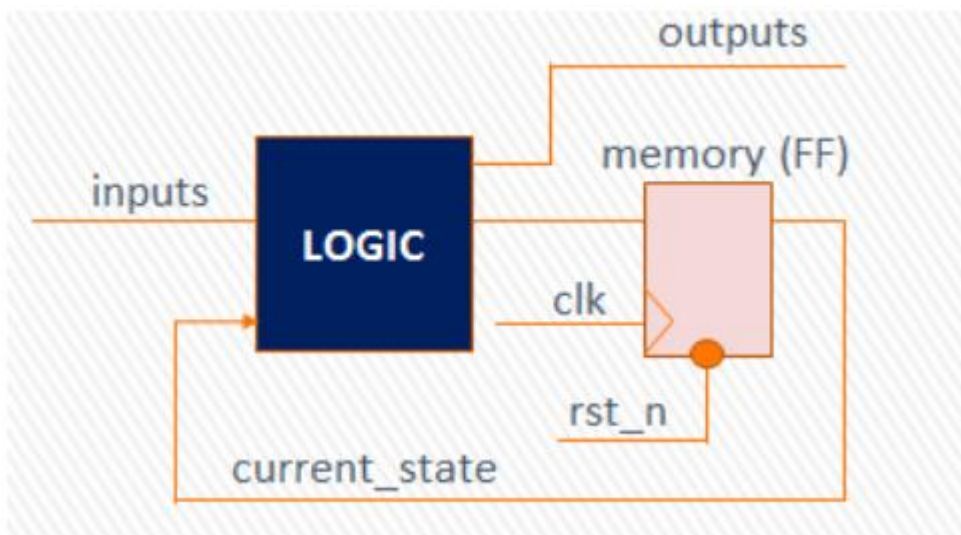
Ứng dụng rộng rãi: Máy trạng thái Mealy thường được sử dụng trong các hệ thống điều khiển, giao diện người dùng, và các ứng dụng yêu cầu phản ứng nhanh với tín hiệu đầu vào.

Ví dụ ứng dụng:

Thiết kế bộ điều khiển cho các hệ thống nhúng.

Xử lý tín hiệu trong các hệ thống truyền thông.

Behavioral Verilog phù hợp để mô tả hành vi và logic điều khiển của mạch, đặc biệt trong giai đoạn mô phỏng và kiểm tra. Dataflow Verilog thích hợp để mô tả các mạch logic đơn giản dựa trên luồng dữ liệu. Máy trạng thái Mealy là một công cụ mạnh mẽ để thiết kế các hệ thống điều khiển phản ứng nhanh với tín hiệu đầu vào.



Hình 12: Máy trạng thái Mealy

❖ *Máy trạng thái Moore.*

Máy trạng thái Moore là một loại máy trạng thái hữu hạn (Finite State Machine - FSM) trong đó tín hiệu đầu ra chỉ phụ thuộc vào trạng thái hiện tại của máy trạng thái, không phụ thuộc vào tín hiệu đầu vào. Đây là điểm khác biệt chính so với máy trạng thái Mealy, nơi tín hiệu đầu ra phụ thuộc vào cả trạng thái hiện tại và tín hiệu đầu vào.

**Đặc điểm của máy trạng thái Moore:*

Tín hiệu đầu ra chỉ phụ thuộc vào trạng thái hiện tại:

Trong máy trạng thái Moore, tín hiệu đầu ra được xác định bởi trạng thái hiện tại của máy trạng thái.

Điều này làm cho máy trạng thái Moore dễ dự đoán và dễ thiết kế hơn so với máy trạng thái Mealy.

Phản ứng chậm hơn so với máy trạng thái Mealy:

Vì tín hiệu đầu ra chỉ thay đổi khi trạng thái thay đổi, máy trạng thái Moore có thể phản ứng chậm hơn so với máy trạng thái Mealy trong một số trường hợp.

Dễ thiết kế và kiểm tra:

Do tín hiệu đầu ra chỉ phụ thuộc vào trạng thái, máy trạng thái Moore thường dễ thiết kế và kiểm tra hơn.

**Cấu trúc của máy trạng thái Moore:*

Máy trạng thái Moore bao gồm các thành phần chính sau:

Tập hợp các trạng thái (State):

Đại diện cho các trạng thái hoạt động của mạch.

Mỗi trạng thái có một giá trị đầu ra tương ứng.

Tín hiệu đầu vào (Input):

Là các tín hiệu được cung cấp cho máy trạng thái để xác định trạng thái tiếp theo.

Tín hiệu đầu ra (Output):

Là các tín hiệu được tạo ra từ trạng thái hiện tại của máy trạng thái.

Luật chuyển đổi trạng thái (Transition Logic):

Xác định cách máy trạng thái chuyển từ trạng thái hiện tại sang trạng thái tiếp theo dựa trên tín hiệu đầu vào.

**Mô tả máy trạng thái Moore:*

Máy trạng thái Moore có thể được mô tả bằng:

Bảng trạng thái (State Table): Liệt kê các trạng thái, tín hiệu đầu vào, trạng thái tiếp theo và tín hiệu đầu ra.

Biểu đồ trạng thái (State Diagram): Một đồ thị biểu diễn các trạng thái và các chuyển đổi giữa chúng.

Ưu điểm của máy trạng thái Moore:

Dễ thiết kế và kiểm tra: Do tín hiệu đầu ra chỉ phụ thuộc vào trạng thái, máy trạng thái Moore thường dễ thiết kế và kiểm tra hơn.

Phù hợp cho các hệ thống yêu cầu độ ổn định cao: Vì tín hiệu đầu ra không thay đổi liên tục theo tín hiệu đầu vào, máy trạng thái Moore phù hợp cho các hệ thống yêu cầu độ ổn định cao.

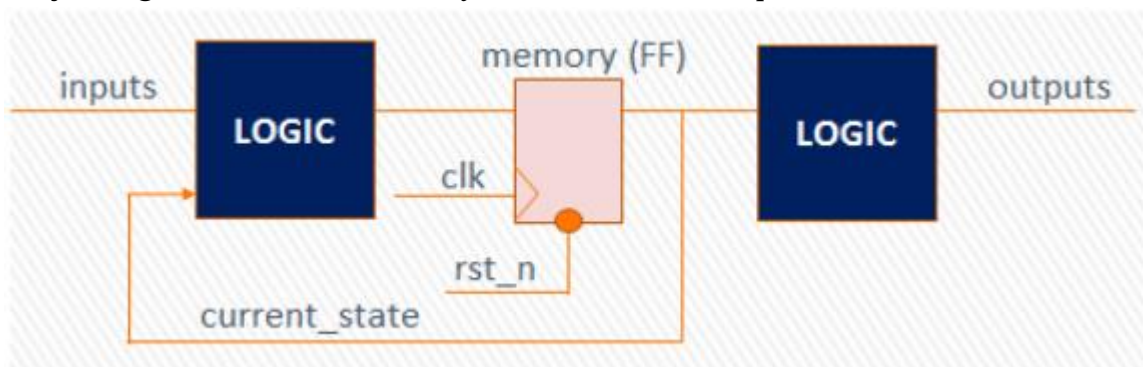
Nhược điểm của máy trạng thái Moore:

Phản ứng chậm hơn so với máy trạng thái Mealy: Do tín hiệu đầu ra chỉ thay đổi khi trạng thái thay đổi, máy trạng thái Moore có thể phản ứng chậm hơn trong một số trường hợp.

Yêu cầu nhiều trạng thái hơn: Để đạt được cùng một chức năng, máy trạng thái Moore có thể yêu cầu nhiều trạng thái hơn so với máy trạng thái Mealy.

Kết luận:

Máy trạng thái Moore là một công cụ mạnh mẽ trong thiết kế các hệ thống số, đặc biệt là các hệ thống yêu cầu độ ổn định và dễ dự đoán cao. Với cấu trúc đơn giản và dễ thiết kế, máy trạng thái Moore được sử dụng rộng rãi trong nhiều lĩnh vực, từ vi xử lý đến hệ thống nhúng và xử lý tín hiệu số. Tuy nhiên, tùy thuộc vào yêu cầu cụ thể của ứng dụng, người thiết kế có thể lựa chọn giữa máy trạng thái Moore và Mealy để đạt được hiệu quả tối ưu.



Hình 13: Máy trạng thái Moore

9 Tổng kết chương

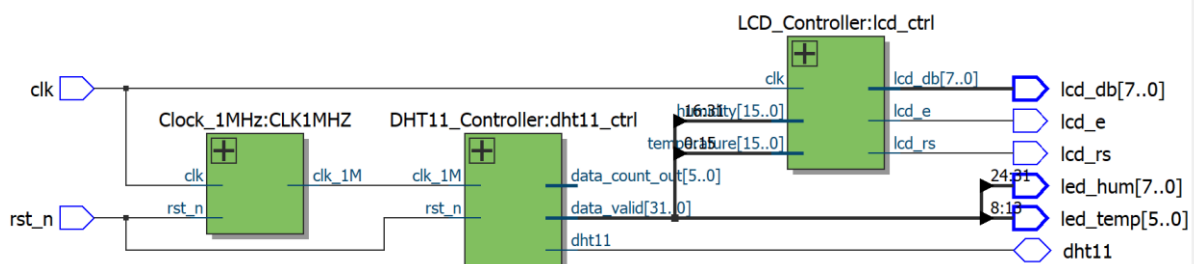
Chương này đã trình bày tổng quan về ngôn ngữ mô tả phần cứng HDL, mạch tích hợp khả trình FPGA, các loại mô hình Verilog bao gồm Structural Verilog, Behavioral Verilog và Dataflow Verilog cũng như hai loại máy trạng thái là Mealy và Moore. Ngoài ra, phần mềm lập trình Verilog Quartus II 13.1 và KIT DeO Nano được sử dụng để thực hiện bài tập lớn này cũng đã được đề cập.

Các kiến thức và công cụ này tạo nền tảng quan trọng cho việc thiết kế và triển khai hệ thống đèn giao thông, từ việc mô tả logic bằng ngôn ngữ Verilog đến việc mô phỏng và kiểm tra trên phần mềm Quartus II, cũng như triển khai thực tế trên KIT DeO Nano. Những nội dung này sẽ được áp dụng cụ thể trong các chương tiếp theo để hoàn thiện đề tài.

CHƯƠNG 2: THIẾT KẾ VÀ XÂY DỰNG MÔ HÌNH HỆ THỐNG

1. Mô hình hệ thống

1.1 Sơ đồ hệ thống



Hình 14: Sơ đồ nguyên lý

1.2 Nguyên lý hoạt động

Sơ đồ nguyên lý này mô tả một hệ thống đo lường và hiển thị nhiệt độ cùng độ ẩm sử dụng cảm biến DHT11 kết hợp với màn hình LCD 16x2. Hệ thống gồm ba khối chính: Clock_1MHz, DHT11_Controller và LCD_Controller, mỗi khối đảm nhận một nhiệm vụ cụ thể trong quá trình hoạt động.

Khối Clock_1MHz có nhiệm vụ tạo xung nhịp 1MHz từ tín hiệu clock đầu vào của FPGA. Do FPGA thường sử dụng clock tần số cao (khoảng 50MHz), khối này sẽ chia tần số xuống 1MHz để cung cấp tín hiệu clock phù hợp cho khối DHT11_Controller.

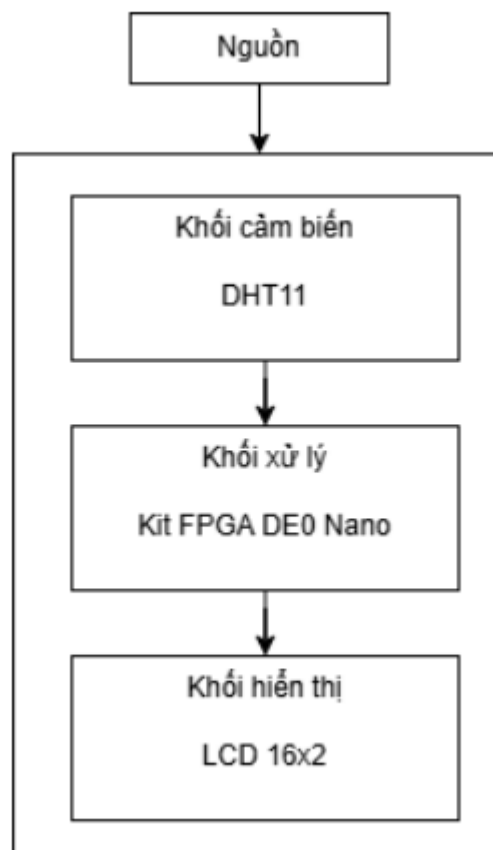
Khối DHT11_Controller đóng vai trò giao tiếp với cảm biến DHT11 để đo nhiệt độ và độ ẩm. Khi nhận được tín hiệu clock 1MHz và tín hiệu reset từ hệ thống, khối này sẽ gửi tín hiệu yêu cầu (start signal) đến cảm biến DHT11 để bắt đầu quá trình đọc dữ liệu. Cảm biến sẽ phản hồi bằng cách gửi 40-bit dữ liệu gồm: 8-bit độ ẩm nguyên, 8-bit độ ẩm thập phân, 8-bit nhiệt độ nguyên, 8-bit nhiệt độ thập phân và 8-bit checksum để kiểm tra tính hợp lệ. Khối DHT11_Controller sẽ xử lý dữ liệu nhận được, kiểm tra checksum để đảm bảo dữ liệu hợp lệ và xuất giá trị nhiệt độ, độ ẩm ra ngoài cùng tín hiệu data_valid để xác nhận dữ liệu có thể sử dụng.

Sau khi dữ liệu đã được xử lý, khối LCD_Controller sẽ nhận giá trị nhiệt độ và độ ẩm từ DHT11_Controller để hiển thị lên màn hình LCD 16x2. Khối này kiểm tra tín hiệu data_valid trước khi thực hiện hiển thị, sau đó chuyển đổi dữ liệu sang dạng ký tự ASCII để hiển thị trên LCD. Đồng thời, khối này cũng điều khiển màn hình LCD thông qua các tín hiệu lcd_db[7:0] (dữ liệu hiển thị), lcd_e (bật màn hình) và lcd_rs (chọn thanh ghi), giúp thông tin được hiển thị một cách rõ ràng. Hệ thống còn được trang bị các LED debug để hỗ trợ kiểm tra hoạt động. Các LED này hiển thị giá trị nhiệt độ và độ ẩm giúp người dùng có thể dễ dàng theo dõi trạng thái hệ thống:

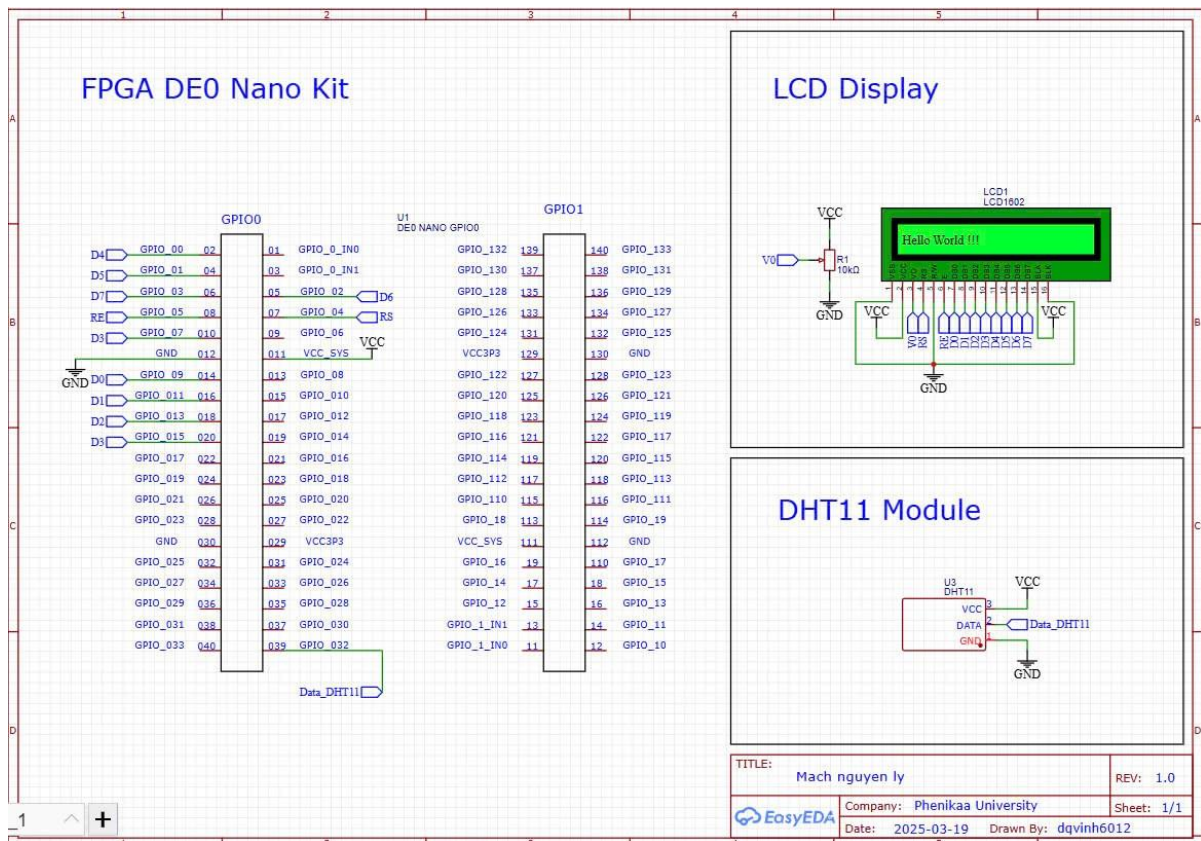
- led_hump[7:0] hiển thị độ ẩm.
- led_temp[5:0] hiển thị nhiệt độ.

Toàn bộ quy trình hoạt động được thực hiện đồng bộ theo xung clock, đảm bảo dữ liệu được xử lý và hiển thị một cách chính xác và liên tục theo thời gian thực.

Sơ đồ khối hệ thống



Hình 15: Sơ đồ khối hệ thống



Hình 16: Mạch nguyên lý

1. Khối nguồn

Chức năng:

Cung cấp điện áp hoạt động cho toàn bộ hệ thống.

Nguyên lý hoạt động:

Nguồn điện cấp chính: Hệ thống sử dụng nguồn 5V được cung cấp từ FPGA DE0-Nano.

Cấp nguồn cho các linh kiện:

Cảm biến DHT11 hoạt động ở mức điện áp 3.3V - 5V. Trong hệ thống này, nó được cấp nguồn 5V từ FPGA.

Màn hình LCD 16x2 cũng hoạt động với điện áp 5V.

Điều chỉnh và ổn định điện áp:

FPGA có bộ điều chỉnh điện áp nội bộ để đảm bảo điện áp hoạt động ổn định.

Cần kiểm tra dòng điện tiêu thụ của các thiết bị để tránh quá tải.

2. Khối cảm biến (DHT11)

Chức năng:

Đo nhiệt độ và độ ẩm của môi trường xung quanh.

Gửi dữ liệu đo được về FPGA để xử lý.

Nguyên lý hoạt động:

1. Khởi tạo cảm biến:

FPGA gửi tín hiệu yêu cầu (start signal) bằng cách kéo mức logic xuống thấp (LOW) trong 18ms rồi đưa lên cao (HIGH) trong 20-40 μ s.

DHT11 nhận tín hiệu này và chuẩn bị gửi dữ liệu.

2. Gửi phản hồi:

DHT11 gửi tín hiệu phản hồi bằng cách kéo xuống mức LOW 80 μ s, sau đó đưa lên mức HIGH 80 μ s.

3. Truyền dữ liệu (40-bit)

Dữ liệu được gửi nối tiếp theo thứ tự:

8-bit độ ẩm nguyên

8-bit độ ẩm thập phân

8-bit nhiệt độ nguyên

8-bit nhiệt độ thập phân

8-bit checksum (kiểm tra lỗi)

4. Cách đọc từng bit dữ liệu:

Một bit '0': LOW (50 μ s) $\rightarrow$ HIGH (26-28 μ s)

Một bit '1': LOW (50 μ s) $\rightarrow$ HIGH (70 μ s)

5. Kiểm tra tính hợp lệ:

FPGA tính toán checksum bằng cách cộng 4 byte đầu (8-bit độ ẩm nguyên, 8-bit độ ẩm thập phân, 8-bit nhiệt độ nguyên, 8-bit nhiệt độ thập phân) và so sánh với byte checksum.

Nếu kết quả trùng khớp, dữ liệu hợp lệ. Nếu không, FPGA sẽ yêu cầu DHT11 gửi lại dữ liệu.

3. Khối xử lý (Kit FPGA DE0-Nano)

Chức năng:

Nhận dữ liệu từ DHT11, kiểm tra lỗi, chuyển đổi dữ liệu và điều khiển hiển thị LCD.

Nguyên lý hoạt động:

Nhận dữ liệu từ DHT11:

FPGA sử dụng bộ đếm thời gian (timer) để đo độ dài của tín hiệu HIGH và phân biệt bit '0' hay '1'.

Kiểm tra dữ liệu:

Kiểm tra checksum để xác nhận dữ liệu nhận được có chính xác hay không.

Nếu sai, FPGA sẽ bỏ qua dữ liệu và yêu cầu DHT11 gửi lại.

Chuyển đổi dữ liệu:

Dữ liệu nhận được ở dạng nhị phân, cần chuyển đổi sang số thập phân để hiển thị.

Điều khiển hiển thị LCD:

Gửi lệnh khởi tạo LCD.

Chuyển đổi số sang dạng chuỗi để gửi đến LCD.

Cập nhật màn hình hiển thị mỗi khi có dữ liệu mới.

4. Khởi hiển thị (LCD 16x2)

Chức năng:

Hiển thị giá trị nhiệt độ và độ ẩm đo được từ cảm biến DHT11.

Nguyên lý hoạt động:

Khởi tạo LCD:

Gửi lệnh thiết lập chế độ giao tiếp 8-bit.

Cấu hình hiển thị (bật/tắt con trỏ, chế độ dịch chuyển...).

Hiển thị dữ liệu:

FPGA chuyển đổi dữ liệu nhiệt độ, độ ẩm từ dạng số sang chuỗi ký tự ASCII.

Gửi từng ký tự một đến LCD theo giao thức 8-bit.

Cập nhật dữ liệu:

Khi có dữ liệu mới từ DHT11, FPGA sẽ xóa màn hình và cập nhật giá trị mới.

Màn hình hiển thị theo định dạng:

2.2.1. Module DHT11_Controller

Nguyên lý hoạt động của module DHT11_Controller:

Module này điều khiển giao tiếp với cảm biến DHT11 để lấy dữ liệu

nhiệt độ và độ ẩm, sau đó xuất giá trị hợp lệ ra *data_valid*. Module hoạt động

theo từng trạng thái cụ thể để đảm bảo việc giao tiếp diễn ra chính xác, đồng thời kiểm tra tính hợp lệ của dữ liệu nhận được trước khi đưa vào sử dụng.

Các bước hoạt động chính:

- ❖ Bước 1: Khởi tạo giao tiếp: Kéo chân dữ liệu xuống mức thấp trong 20ms để báo hiệu cảm biến gửi dữ liệu.
- ❖ Bước 2: Nhận phản hồi: Chờ cảm biến phản hồi và kiểm tra tín hiệu xác nhận.
- ❖ Bước 3: Đọc dữ liệu: Nhận 40 bit dữ liệu bao gồm độ ẩm nguyên, độ ẩm thập phân, nhiệt độ nguyên, nhiệt độ thập phân, và checksum.
- ❖ Bước 4: Kiểm tra checksum: Đảm bảo dữ liệu nhận được hợp lệ.
- ❖ Bước 5: Lưu trữ dữ liệu: Xuất dữ liệu hợp lệ ra đầu ra *data_valid*

```
module DHT11_Controller (
    input          clk_1M,           // Clock 50MHz
    input          rst_n,            // Reset active-low
    inout          dht11,            // DHT11 data pin
    output reg [31:0] data_valid,    // Output temperature & humidity (valid data)
    output wire [5:0] data_count_out // Xuất số bit nhận được
);
```

Giải thích:

clk: Clock đầu vào có tần số 50MHz.

rst_n: Tín hiệu reset chủ động mức thấp.

dht11: Chân dữ liệu của cảm biến DHT11, được thiết lập là inout vì module vừa ghi vừa đọc dữ liệu từ cảm biến.

data_valid: Xuất dữ liệu hợp lệ (nhiệt độ và độ ẩm) nếu checksum hợp lệ.

data_count_out: Xuất số bit dữ liệu đã nhận từ DHT11.

❖ Khai báo các tham số (Parameter)

```
parameter START_READ          = 1000_000; //READ data each 1 second
parameter CONTROL_LOW         = 0;
parameter CONTROL_LOW_20MS    = 1;
parameter CONTROL_HIGH_13US   = 2;
parameter READ_LOW_83US       = 3;
parameter READ_HIGH_87US      = 4;
parameter READ_DATA            = 5;
parameter KEEP_DELAY          = 6;
```

Giải thích

Module DHT11_Controller sử dụng các tham số để xác định thời gian chờ và điều khiển các trạng thái giao tiếp với cảm biến DHT11.

1. START_READ = 1000000: Xác định khoảng thời gian giữa hai lần đọc dữ liệu (1 giây).
2. CONTROL_LOW = 0: Trạng thái chờ, giữ chân dht11 ở mức cao.
3. CONTROL_LOW_20MS = 1: Kéo dht11 xuống thấp 20ms để kích hoạt cảm biến.
4. CONTROL_HIGH_13US = 2: Đưa dht11 lên cao 13 μ s để chuyển quyền điều khiển.
5. READ_LOW_83US = 3: Cảm biến phản hồi bằng mức thấp 83 μ s trước khi gửi dữ liệu.
6. READ_HIGH_87US = 4: Cảm biến giữ mức cao 87 μ s để chuẩn bị gửi dữ liệu.
7. READ_DATA = 5: Nhận 40 bit dữ liệu (nhiệt độ & độ ẩm) từ DHT11.
8. KEEP_DELAY = 6: Giữ trạng thái chờ 2 giây trước khi thực hiện lần đo tiếp theo.

❖ Định nghĩa các thanh ghi (Register)

```
reg [2:0] cur_state, next_state;
reg [20:0] count_lus;
reg [5:0] data_count;
reg [39:0] data_temp;
reg [4:0] clk_cnt;
reg us_clear;
reg state;
reg dht_buffer;
reg dht_d0, dht_d1;
```

Giải thích

cur_state, next_state: Trạng thái hiện tại và trạng thái tiếp theo của FSM.

count_1us: Bộ đếm số chu kỳ microsecond (1MHz).

data_count: Đếm số bit nhận được từ DHT11.

data_temp: Lưu trữ 40-bit dữ liệu từ DHT11.

clk_cnt: Bộ đếm dùng để chia clock từ 50MHz xuống 1MHz.

us_clear: Tín hiệu reset bộ đếm count_1us.

state: Trạng thái trong quá trình nhận bit (0 hoặc 1).

dht_buffer: Lưu trữ dữ liệu điều khiển đầu ra của dht11.

dht_d0, dht_d1: Dùng để phát hiện cạnh lên/xuống của tín hiệu DHT11.

❖ Khai báo Wire

```
/****** Wire Define *****/  
wire dht_podge, dht_nedge;  
  
assign dht11 = dht_buffer;  
assign dht_podge = ~dht_d1 & dht_d0; // Posedge detect  
assign dht_nedge = dht_d1 & ~dht_d0; // Negedge detect  
assign data_count_out = data_count;
```

Giải thích

dht_podge: Phát hiện cạnh lên của tín hiệu DHT11.

dht_nedge: Phát hiện cạnh xuống của tín hiệu DHT11.

data_count_out: Xuất số bit đã nhận.

Bộ chia clock 1MHz

```

/***** 1MHz Clock Generator *****/
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        clk_cnt <= 5'd0;
        clk_1M <= 1'b0;
    end
    else if (clk_cnt < 5'd24)
        clk_cnt <= clk_cnt + 1'b1;
    else begin
        clk_cnt <= 5'd0;
        clk_1M <= ~clk_1M;
    end
end
end

```

Vì cảm biến DHT11 yêu cầu thời gian chính xác trong khoảng micro giây (μ s), một bộ chia tần số được sử dụng để tạo ra tín hiệu xung có tần số 1MHz từ clock hệ thống 25MHz.

Cơ chế hoạt động:

- Một bộ đếm (clk_cnt) tăng dần từ 0 đến 24.
- Khi đạt giá trị 24, bộ đếm reset và đảo trạng thái của clk_1M.
- Kết quả là tín hiệu clk_1M có chu kỳ 1 μ s, tương ứng với tần số 1MHz

Bộ đếm 1us

```

/***** 1us Counter *****/
always @(posedge clk_1M or negedge rst_n) begin
    if (!rst_n) begin
        count_1us <= 21'd0;
        dht_d0 <= 1'b1;
        dht_d1 <= 1'b1;
    end
    else if (us_clear) begin
        count_1us <= 21'd0;
    end
    else begin
        dht_d0 <= dht11;
        dht_d1 <= dht_d0;
        count_1us <= count_1us + 1'b1;
    end
end
end

```

Sử dụng xung nhịp 1 MHz (clk_1M) để đếm thời gian với độ chính xác 1 μ s và xử lý tín hiệu từ cảm biến DHT11.

- Khi rst_n = 0, hệ thống reset: bộ đếm count_1us = 0, tín hiệu dht_d0, dht_d1 được đặt về mức cao (1'b1).

- Khi `us_clear = 1`, bộ đếm `count_1us` được xóa về 0.
- Ở mỗi chu kỳ xung nhịp (1 μ s):
 - `dht_d0` cập nhật giá trị từ cảm biến `dht11`,
 - `dht_d1` lưu giá trị trước đó của `dht_d0`, giúp tạo độ trễ một chu kỳ,
 - `count_1us` tăng dần, dùng để đo thời gian.

❖ FSM điều khiển giao tiếp DHT11

FSM (Finite State Machine – Máy trạng thái hữu hạn) là thành phần chính trong module `DHT11_Controller`, chịu trách nhiệm thực hiện quy trình giao tiếp với DHT11 theo các bước chuẩn hóa.

DHT11 sử dụng giao thức one-wire, trong đó MCU (Vi điều khiển) điều khiển quá trình đọc dữ liệu thông qua việc thay đổi trạng thái của chân dữ liệu. Mỗi trạng thái tương ứng với một bước trong giao thức giao tiếp DHT11.

Kiểm tra cạnh lên/xuống của tín hiệu để đồng bộ dữ liệu.

Xử lý và lưu bit 0 hoặc bit 1 dựa trên độ dài xung cao.

• Quy trình giao tiếp với DHT11

- Bước 1: Khởi tạo giao tiếp: MCU giữ chân dữ liệu mức thấp 20ms để yêu cầu cảm biến gửi dữ liệu. Sau đó, MCU thả nổi chân dữ liệu (1'bz) để DHT11 có thể điều khiển tín hiệu.

- Bước 2: Phản hồi từ DHT11: DHT11 kéo mức thấp 83 μ s rồi mức cao 87 μ s để xác nhận phản hồi.

- Bước 3: Gửi dữ liệu: DHT11 gửi 40 bit dữ liệu theo thứ tự: 8 bit độ ẩm (phần nguyên), 8 bit độ ẩm (phần thập phân), 8 bit nhiệt độ (phần nguyên), 8 bit nhiệt độ (phần thập phân), 8 bit checksum

- Bước 4: Kiểm tra checksum: Kiểm tra tổng các byte dữ liệu có khớp với checksum không.

Cụ thể: FSM được định nghĩa trong hai always block chính là:

- Cập nhật trạng thái hiện tại (`cur_state`)

```

|
|/***** State Machine *****/
|always @(posedge clk_1M or negedge rst_n) begin
|    if (!rst_n)
|        cur_state <= CONTROL_LOW;
|    else
|        cur_state <= next_state;
|end

```

Khi hệ thống reset ($\text{rst_n} = 0$), trạng thái ban đầu là `CONTROL_LOW`.

Khi có xung dương `clk_1M`, `cur_state` sẽ nhận giá trị của `next_state`, tức là chuyển sang trạng thái kế tiếp theo quy tắc FSM.

```

|always @(posedge clk_1M or negedge rst_n) begin
|    if(!rst_n) begin
|        next_state <= CONTROL_LOW;
|        dht_buffer <= 1'bz;
|        state <= 1'b0;
|        us_clear <= 1'b0;
|        data_temp <= 40'd0;
|        data_count <= 6'd0;
|    end

```

- Nếu reset, đặt tất cả các giá trị về trạng thái ban đầu:

- `next_state = CONTROL_LOW`: Bắt đầu từ trạng thái cấp nguồn.
- `dht_buffer = 1'bz`: Chân dữ liệu DHT11 ở chế độ thả nổi.
- `state = 0`: Trạng thái nhận bit ban đầu.
- `data_temp = 40'd0`: Xóa bộ nhớ lưu trữ dữ liệu.
- `data_count = 0`: Chưa có bit nào nhận được.

❖ Trạng thái `CONTROL_LOW` – Chờ khởi động:

```

----
else begin
    case (cur_state)
        CONTROL_LOW: begin
            if(count_lus < START_READ) begin
                dht_buffer <= 1'bz;
                us_clear    <= 1'b0;
            end else begin
                next_state <= CONTROL_LOW_20MS;
                us_clear    <= 1'b1;
            end
        end
    end
end

```

- Chờ thời gian khởi động START_READ = 1,000,000 (tương đương 1 giây).
- Giữ chân dht11 ở trạng thái thả nổi (1'bz).
- Sau khi chờ đủ 1 giây, chuyển sang trạng thái CONTROL_LOW_20MS.
- ❖ Trạng thái CONTROL_LOW_20MS – Gửi tín hiệu khởi tạo

```

CONTROL_LOW_20MS: begin
    if(count_lus < 20000) begin
        dht_buffer <= 1'b0;
        us_clear    <= 1'b0;
    end else begin
        next_state <= CONTROL_HIGH_13US;
        dht_buffer <= 1'bz;
        us_clear    <= 1'b1;
    end
end
end

```

MCU kéo chân dht11 xuống thấp trong 20ms để yêu cầu DHT11 gửi dữ liệu.

- Sau đó, thả nổi chân dht11 (1'bz) và chuyển sang CONTROL_HIGH_13US
- ❖ Trạng thái CONTROL_HIGH_13US – Chờ phản hồi từ DHT11

```

94 | CONTROL_HIGH_13US: begin
95 |     if (count_lus < 20) begin
96 |         us_clear <= 1'b0;
97 |         if (dht_nedge) begin
98 |             next_state <= READ_LOW_83US;
99 |             us_clear <= 1'b1;
100 |         end
101 |     end else begin
102 |         next_state <= KEEP_DELAY;
103 |     end
104 | end
105 |

```

MCU chờ cạnh xuống (dht_nedge) của DHT11.

- Nếu không thấy cạnh xuống sau 20 μ s, chuyển sang KEEP_DELAY.

Trạng thái READ_LOW_83US – Xác nhận phản hồi

```

114 |
115 | S_LOW_83US: begin
116 |     if (dht_podge)
117 |         next_state <= S_HIGH_87US;
118 |     end

```

Khi DHT11 giữ mức thấp 83 μ s, MCU chờ cạnh lên (dht_podge) để tiếp tục.

- ❖ Trạng thái READ_HIGH_87US – Chuẩn bị nhận dữ liệu

```

110 |
111 | READ_HIGH_87US: begin
112 |     if (dht_nedge) begin
113 |         next_state <= READ_DATA;
114 |         us_clear <= 1'b1;
115 |     end else begin
116 |         data_count <= 6'd0;
117 |         data_temp <= 40'd0;
118 |         state <= 1'b0;
119 |     end
120 | end
121 |

```

Khi phát hiện cạnh xuống (dht_nedge), chuyển sang trạng thái nhận dữ liệu.

- Xóa dữ liệu cũ (data_count = 0, data_temp = 0).

- ❖ Trạng thái READ_DATA – Nhận dữ liệu

```

121 |
122 | READ_DATA: begin
123 |     case (state)
124 |     0: begin
125 |         if (dht_podge) begin
126 |             state <= 1'b1;
127 |             us_clear <= 1'b1;
128 |         end else begin
129 |             us_clear <= 1'b0;
130 |         end
131 |     end
132 |     1: begin
133 |         if (dht_nedge) begin
134 |             data_count <= data_count + 1'b1;
135 |             state <= 1'b0;
136 |             us_clear <= 1'b1;
137 |             if (count_lus < 60)
138 |                 data_temp <= {data_temp[38:0], 1'b0};
139 |             else
140 |                 data_temp <= {data_temp[38:0], 1'b1};
141 |             end else begin
142 |                 us_clear <= 1'b0;
143 |             end
144 |         end
145 |     endcase
146 |
147 |     if (data_count == 40) begin
148 |         next_state <= KEEP_DELAY;
149 |     end
150 | end
151 |

```

Chờ cạnh lên để bắt đầu nhận bit.

- Chờ cạnh xuống để xác định bit 0 hoặc bit 1:

- Nếu xung cao < 60μs, ghi 0.

- Nếu xung cao ≥ 60μs, ghi 1.

- Nhận đủ 40 bit, chuyển sang KEEP_DELAY.

❖ Trạng thái KEEP_DELAY – Chờ trước khi lấy dữ liệu mới

```

151 |
152 | KEEP_DELAY: begin
153 |     if (count_lus < 2000_000)
154 |         us_clear <= 1'b0;
155 |     else begin
156 |         next_state <= CONTROL_LOW_20MS;
157 |         us_clear <= 1'b1;
158 |     end
159 | end
160 |
161 | default: next_state <= CONTROL_LOW;
162 | endcase
163 | end
164 | end
165 |

```

- Chờ 2 giây trước khi đọc dữ liệu tiếp theo.

```

174
175  /***** Checksum Verification *****/
176  always @(posedge clk_1M or negedge rst_n) begin
177      if (!rst_n) begin
178          data_valid <= 32'd0;
179      end
180      else if (data_count == 40) begin
181          reg [7:0] humidity_int, humidity_dec;
182          reg [7:0] temperature_int, temperature_dec;
183          reg [7:0] checksum, checksum_calc;
184
185          humidity_int    = data_temp[39:32];
186          humidity_dec    = data_temp[31:24];
187          temperature_int = data_temp[23:16];
188          temperature_dec = data_temp[15:8];
189          checksum        = data_temp[7:0];
190
191          checksum_calc = humidity_int + humidity_dec + temperature_int + temperature_dec;
192
193          if (checksum == checksum_calc) begin
194              data_valid <= data_temp[39:8];
195          end
196          else begin
197              data_valid <= 32'd0;
198          end
199      end
200  end
201

```

- Tách các byte dữ liệu nhiệt độ, độ ẩm.

Tính tổng kiểm tra checksum_calc và so sánh với checksum từ DHT11.

- Nếu kiểm tra hợp lệ, xuất dữ liệu hợp lệ ra data_valid.

```

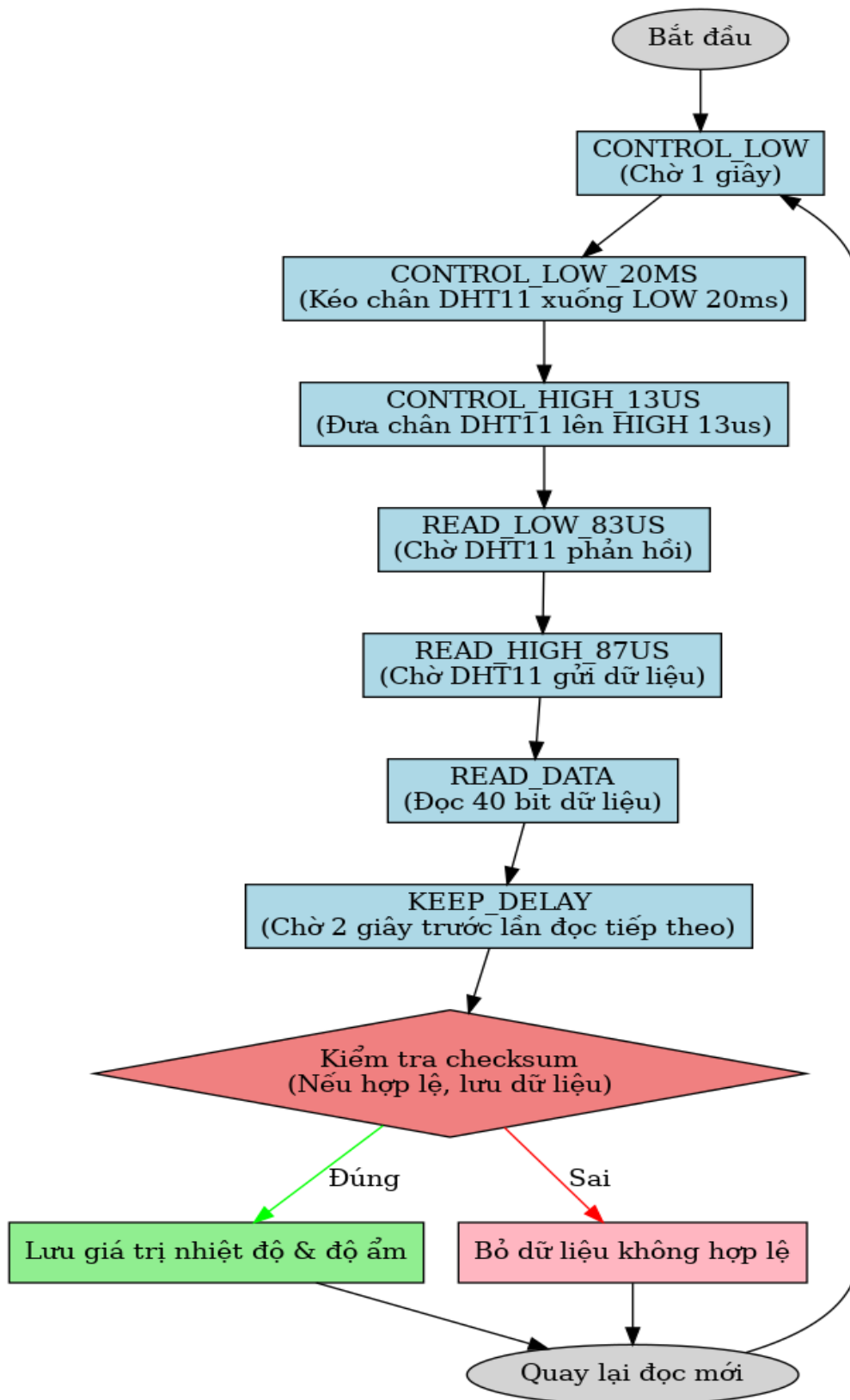
202
203  /***** Edge Detection *****/
204  always @(posedge clk_1M or negedge rst_n) begin
205      if (!rst_n) begin
206          dht_d0 <= 1'b1;
207          dht_d1 <= 1'b1;
208      end
209      else begin
210          dht_d0 <= dht11;
211          dht_d1 <= dht_d0;
212      end
213  end
214
215  endmodule
216

```

Do DHT11 truyền dữ liệu bằng cách thay đổi mức điện áp trên chân dữ liệu, cần một bộ phát hiện cạnh để xác định thời điểm chuyển mức:

- Cạnh lên (Posedge): Khi tín hiệu thay đổi từ 0 → 1.
- Cạnh xuống (Negedge): Khi tín hiệu thay đổi từ 1 → 0.

Hai thanh ghi dht_d0 và dht_d1 được sử dụng để ghi nhận trạng thái trước đó của chân dữ liệu, từ đó xác định các cạnh lên và xuống.



Hình 17: Sơ đồ thuật toán module DHT11

2.2.3.Module Clock_1MHz

```
1 // Create clock divider from 50MHz to 1MHz
2 module Clock_1MHz(clk, rst_n, clk_1M);
3     //wire clk and reset input
4     input clk, rst_n;
5     //output clock scale to 1MHz
6     output clk_1M;
7     //clk_cnt register to count from 1 to 25 -> create 50% pulse width 1MHz clock
8     reg [4:0] clk_cnt;
9     reg clk_1M;
10
```

Đầu vào (clk): Nhận xung clock 50 MHz từ hệ thống.

Đầu vào (rst_n): Reset hệ thống khi ở mức thấp (active-low reset).

Đầu ra (clk_1M): Phát ra xung clock có tần số 1 MHz.

Bộ đếm (clk_cnt): Được dùng để theo dõi số xung clock 50 MHz.

Tín hiệu clock (clk_1M): Lưu trạng thái của xung clock 1 MHz.

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        clk_cnt <= 5'd0;
        clk_1M <= 1'b0;
    end
    else if (clk_cnt < 5'd24)
        clk_cnt <= clk_cnt + 1'b1;
    else begin
        clk_cnt <= 5'd0;
        clk_1M <= ~clk_1M;
    end
end
endmodule
```

Reset hệ thống

- Khi reset (rst_n = 0):
 - Bộ đếm clk_cnt được đặt về 0.
 - Tín hiệu clk_1M cũng được đặt về 0.

→ Công dụng: Đảm bảo hệ thống khởi động đúng trạng thái.

Đếm số xung clock

- Mỗi xung 50 MHz, bộ đếm clk_cnt sẽ tăng thêm 1.
- Khi clk_cnt đạt giá trị 24, tín hiệu clock đầu ra sẽ thay đổi.

→ Công dụng: Đếm đủ 25 xung clock 50 MHz để thực hiện chia tần số.

Đảo trạng thái clock đầu ra

- Khi clk_cnt đạt 24, nó sẽ được đặt về 0.
- Tín hiệu clk_1M bị đảo trạng thái (0 → 1 hoặc 1 → 0).

→ Công dụng: Mỗi chu kỳ clk_1M kéo dài $(25 \times 20 \text{ ns}) \times 2 = 1 \mu\text{s}$, tức là 1 MHz.

Phân tích tần số

Tần số clock đầu vào

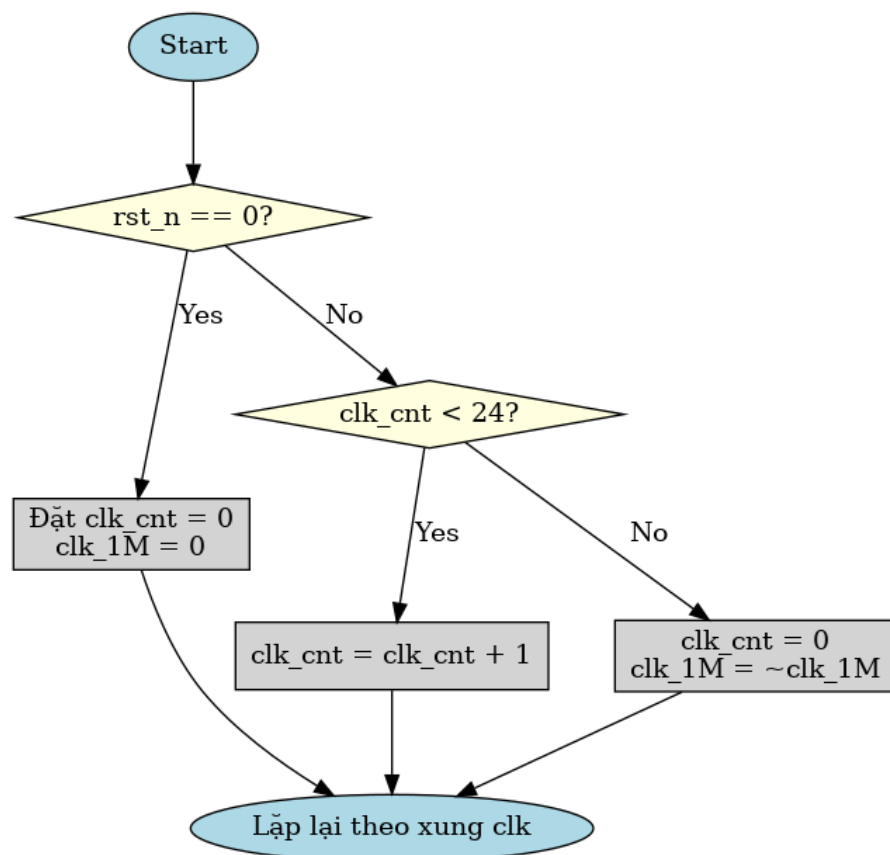
- clk = 50 MHz → Chu kỳ = 20 ns ($1/50 \text{ MHz} = 20 \text{ ns}$).

Số lần đếm trước khi đảo trạng thái

- clk_cnt chạy từ 0 đến 24 (tổng cộng 25 xung clock 50 MHz).
- Mất $25 \times 20 \text{ ns} = 500 \text{ ns}$ để clk_cnt đạt 24.
- Sau đó clk_1M bị đảo trạng thái, tức là mỗi chu kỳ clk_1M = $500 \text{ ns} \times 2 = 1000 \text{ ns} = 1 \mu\text{s}$.

Tần số đầu ra

$$f_{out} = \frac{f_{in}}{2 \times 25} = \frac{50 \text{ MHz}}{50} = 1 \text{ MHz}$$



Hình 18: Lưu đồ thuật toán module Clock_1MHz

2.2.2. Module LCD_Controller

2.2.2.1. Nguyên lý hoạt động của module LCD_Controller

Module LCD_Controller trên được viết bằng Verilog, dùng để điều khiển một màn hình LCD 16x2 hiển thị dữ liệu nhiệt độ và độ ẩm nhận từ cảm biến DHT11 (hoặc cảm biến tương đương). Màn hình LCD sử dụng giao tiếp song song 8-bit, với các chân điều khiển gồm lcd_rs, lcd_e, và bus dữ liệu lcd_db[7:0].

Bước 1: Khởi tạo LCD – Gửi các lệnh thiết lập chế độ hiển thị, bật màn hình, di chuyển con trỏ, và xóa màn hình.

❖ Bước 2: Hiển thị chuỗi cố định – Lưu trữ và gửi chuỗi "Nhiệt Do:", "Do am:" lên LCD.

❖ Bước 3: Chuyển đổi dữ liệu – Tách phần nguyên, phần thập phân và chuyển đổi sang mã ASCII.

❖ Bước 4: Hiển thị dữ liệu – Gửi nhiệt độ, độ ẩm lên LCD thông qua lcd_db, điều khiển bằng lcd_rs và lcd_e.

❖ Bước 5: Cập nhật dữ liệu định kỳ – Sau một khoảng thời gian delay, làm mới giá trị hiển thị.

```
module LCD_Controller (
    input wire clk,
    output reg lcd_rs, lcd_e,
    output reg [7:0] lcd_db,
    input wire [15:0] temperature, // Giá trị nhiệt độ từ cảm biến
    input wire [15:0] humidity      // Giá trị độ ẩm từ cảm biến
);
```

Giải thích:

input wire clk: Clock đầu vào có tần số 25MHz.

input wire reset: Tín hiệu reset chủ động mức thấp.

input wire [7:0] temperature, humidity giá trị nhiệt độ, độ ẩm thu được từ DHT11 output reg lcd_rs, lcd_e

Tín hiệu điều khiển LCD output reg [7:0] lcd_db Bus dữ liệu 8-bit tới LCD

```
wire [7:0] temp_int = temperature[15:8]; // Phần nguyên nhiệt độ
wire [7:0] temp_double = temperature[7:0]; // Phần thập phân nhiệt độ
wire [7:0] hum_int = humidity[15:8]; // Phần nguyên độ ẩm
wire [7:0] hum_double = humidity[7:0]; // Phần thập phân độ ẩm

reg [5:0] state = 0;
```

Chia giá trị nhiệt độ và độ ẩm thành phần nguyên và thập phân

```

// ...
reg [5:0] state = 0;
reg [23:0] counter = 0;
reg [7:0] char_array [0:32];
reg [4:0] char_index = 0;

always @(posedge clk) begin

```

- state: Điều khiển từng bước trong quá trình khởi tạo và hiển thị dữ liệu.
- counter: Bộ đếm để tạo độ trễ giữa các trạng thái.
- char_array: Lưu chuỗi ký tự hiển thị trên LCD.
- char_index: Chỉ số hiện tại trong mảng char_array.

```

always @(posedge clk) begin
    // Cập nhật mảng hiển thị nhiệt độ và độ ẩm
    char_array[0] = "N"; char_array[1] = "h"; char_array[2] = "i";
    char_array[3] = "e"; char_array[4] = "t"; char_array[5] = " ";
    char_array[6] = "d"; char_array[7] = "o"; char_array[8] = ".";
    char_array[9] = " ";
    char_array[10] = (temp_int / 10) + 8'h30; // Chữ số hàng chục
    char_array[11] = (temp_int % 10) + 8'h30; // Chữ số hàng đơn vị
    char_array[12] = 8'h2E;
    char_array[13] = (temp_double / 10) + 8'h30; // Chữ số hàng chục
    char_array[14] = (temp_double % 10) + 8'h30; // Chữ số hàng đơn vị
    char_array[15] = "C";
    char_array[16] = 8'hC0; // Lệnh di chuyển con trỏ
    char_array[17] = "D"; char_array[18] = "o"; char_array[19] = " ";
    char_array[20] = "a"; char_array[21] = "m"; char_array[22] = ".";
    char_array[23] = " ";
    char_array[24] = (hum_int / 10) + 8'h30; // Chữ số hàng chục
    char_array[25] = (hum_int % 10) + 8'h30; // Chữ số hàng đơn vị
    char_array[26] = ".";
    char_array[27] = (hum_double / 10) + 8'h30; // Chữ số hàng chục
    char_array[28] = (hum_double % 10) + 8'h30; // Chữ số hàng đơn vị
    char_array[29] = "%";
    char_array[30] = " "; // Loại bỏ các ký tự thừa ở cuối
    char_array[31] = " ";
    char_array[32] = " ";
end

```

Cập nhật chữ: Gán trực tiếp chuỗi "Nhiệt độ: " cho các phần tử đầu mảng và "Độ ẩm:" cho phần sau.

Chuyển số sang ký tự: Các giá trị số (như temp_int, temp_double, hum_int, hum_double) được chia để lấy chữ số hàng chục và đơn vị, sau đó cộng với 8'h30 để chuyển sang mã ASCII.

Lệnh điều khiển con trỏ: char_array[16] = 8'hC0 dùng để di chuyển con trỏ hiển thị xuống dòng thứ 2.

```

always @(posedge clk) begin
    counter <= counter + 1;
    if (counter[21]) begin // Tạo delay
        case (state)
            // **Khởi tạo LCD**

```

Vòng lặp: chạy liên tục theo xung clk.

Cơ chế tạo delay: Mỗi 2^{21} chu kỳ clock, thực hiện một bước trong state machine. Module tạo delay giữa các bước để đảm bảo LCD nhận lệnh ổn định (vì LCD cần thời gian xử lý sau mỗi lệnh/ký tự)

```
case (state)
// **Khởi tạo LCD**
0: begin lcd_rs <= 0; lcd_db <= 8'h38; lcd_e <= 1; state <= 1; end
1: begin lcd_e <= 0; state <= 2; end
2: begin lcd_db <= 8'h0C; lcd_e <= 1; state <= 3; end
3: begin lcd_e <= 0; state <= 4; end
4: begin lcd_db <= 8'h06; lcd_e <= 1; state <= 5; end
5: begin lcd_e <= 0; state <= 6; end
6: begin lcd_db <= 8'h01; lcd_e <= 1; state <= 7; end
7: begin lcd_e <= 0; state <= 8; end
```

8'h38: Cấu hình LCD ở chế độ 8-bit, 2 dòng.

8'h0C: Bật màn hình, tắt con trỏ.

8'h06: Thiết lập chế độ tự động dịch con trỏ.

8'h01: Xóa màn hình.

```
// **Hiển thị chuỗi Nhiệt độ trên dòng 1 và Độ ẩm trên dòng 2**
8: begin
    if (char_index < 33) begin
        if (char_array[char_index] == 8'hC0) begin
            lcd_rs <= 0; // Lệnh di chuyển con trỏ
        end else begin
            lcd_rs <= 1; // Hiển thị dữ liệu
        end
        lcd_db <= char_array[char_index];
        lcd_e <= 1;
        state <= 9;
    end else begin
        char_index <= 9;
        lcd_rs <= 0;
        state <= 30; // Dừng lại
    end
end
9: begin
    lcd_e <= 0;
    char_index <= char_index + 1;
    state <= 8;
end
```

Gửi từng ký tự từ char_array lên LCD.

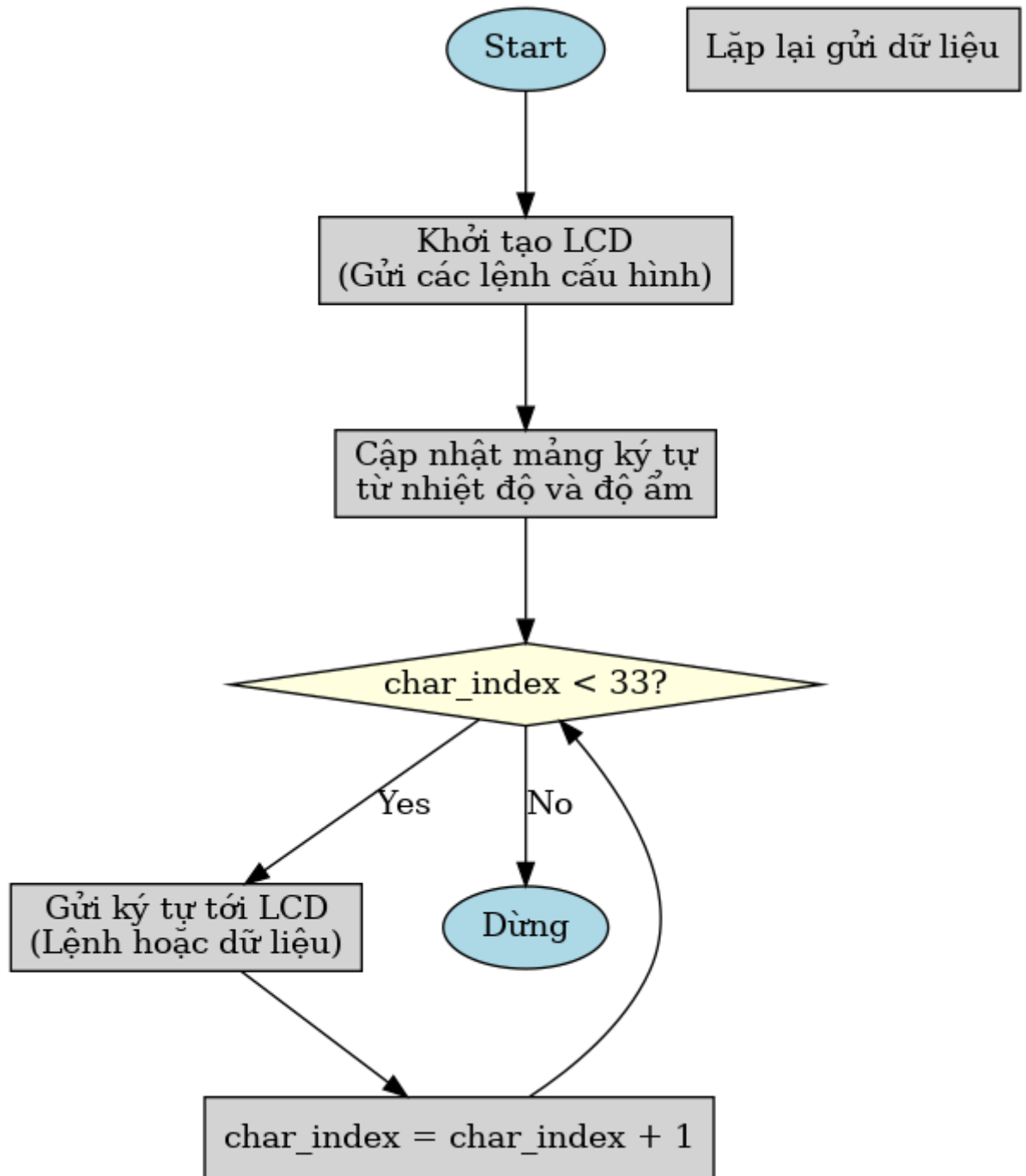
Khi gặp 8'hC0, gửi lệnh chuyển con trỏ xuống dòng 2.

```

30: begin
    state <= 30; // Giữ nguyên
end
endcase
counter <= 0;

```

- Dừng quá trình sau khi hiển thị hoàn tất.



Hình 19: Lưu đồ thuật toán LCD

2.2.3. Module Top_Module

2.2.3.1. Nguyên lý hoạt động của Top_Module

Cấu trúc và nguyên lý hoạt động

Module bao gồm bốn thành phần chính:

Các LED này sẽ sáng theo mức tương ứng với giá trị đo được, giúp dễ dàng kiểm tra trạng thái nhiệt độ và độ ẩm ngay cả khi không có màn hình LCD.

Quá trình hoạt động của hệ thống

1. Khi hệ thống khởi động, bộ tạo xung Clock_1MHz sẽ tạo tín hiệu xung nhịp 1MHz từ xung nhịp hệ thống và cấp cho DHT11_Controller.
2. DHT11_Controller giao tiếp với cảm biến DHT11, gửi tín hiệu khởi động và nhận dữ liệu trả về dưới dạng chuỗi 32-bit.
3. Sau khi lấy được dữ liệu, bộ điều khiển sẽ tách phần nhiệt độ và độ ẩm ra hai biến riêng biệt để xử lý.
4. Dữ liệu nhiệt độ và độ ẩm được truyền đến LCD_Controller, từ đó hiển thị lên màn hình LCD 16x2.
5. Đồng thời, một phần dữ liệu cũng được xuất ra led_hum và led_temp để biểu diễn trực quan giá trị nhiệt độ và độ ẩm bằng cách điều khiển độ sáng của LED.

2. Tổng kết chương

Chương này tập trung vào việc mô tả chi tiết quá trình phân tích yêu cầu đề bài và xây dựng hệ thống đo nhiệt độ, độ ẩm và áp suất sử dụng FPGA DEO Nano. Nhóm đã tiếp cận bài toán bằng cách phân tích kỹ lưỡng các yêu cầu, từ đó đưa ra thiết kế hệ thống đáp ứng đầy đủ các chức năng cần thiết.

Các nội dung chính trong chương:

1. Phân tích yêu cầu đề bài:

Nhóm đã phân tích các yêu cầu về chức năng và hoạt động của hệ thống, bao gồm:

Đo nhiệt độ, độ ẩm và áp suất từ các cảm biến tương ứng.

Hiển thị các giá trị đo được trên màn hình hoặc giao diện người dùng.

Xử lý và chuyển đổi dữ liệu từ cảm biến sang định dạng có thể hiển thị.

Xác định các yêu cầu về độ chính xác và tốc độ đo.

Xác định các đầu vào (input) và đầu ra (output) cụ thể của hệ thống:

Đầu vào: Tín hiệu từ các cảm biến nhiệt độ, độ ẩm và áp suất.

Đầu ra: Dữ liệu nhiệt độ, độ ẩm và áp suất đã được xử lý, tín hiệu điều khiển màn hình hiển thị.

2. Xây dựng hệ thống:

Hệ thống được thiết kế dựa trên các mô hình thiết kế phần cứng số phù hợp, có thể là máy trạng thái (FSM) hoặc các mô hình xử lý dữ liệu song song, tùy thuộc vào yêu cầu về tốc độ và độ phức tạp của hệ thống.

Sử dụng ngôn ngữ mô tả phần cứng Verilog hoặc VHDL để mô tả hành vi và cấu trúc của hệ thống.

Hệ thống được chia thành các module độc lập, bao gồm:

Module đọc dữ liệu từ cảm biến.

Module xử lý và chuyển đổi dữ liệu.

Module điều khiển hiển thị.

Module giao tiếp với người dùng .

Việc chia nhỏ hệ thống thành các module giúp dễ dàng quản lý, kiểm tra và gỡ lỗi.

3. Các trạng thái của hệ thống:

Nhóm đã xác định và mô tả các trạng thái hoạt động của hệ thống, bao gồm:

Trạng thái đọc dữ liệu từ cảm biến.

Trạng thái xử lý dữ liệu.

Trạng thái hiển thị dữ liệu.

Các trạng thái xử lý lỗi (nếu có).

Mỗi trạng thái được liên kết với các tín hiệu đầu vào và đầu ra tương ứng, cũng như các điều kiện chuyển trạng thái.

Khai báo module và các tín hiệu đầu vào, đầu ra

```

1 module Top_Module (
2     input wire clk,
3     input wire rst_n,
4     inout wire dht11,
5     output wire lcd_rs,
6     output wire lcd_e,
7     output wire [7:0] lcd_db,
8     output wire [7:0] led_hum, // LED hiển thị độ ẩm
9     output wire [5:0] led_temp // LED hiển thị nhiệt độ
10 );
11

```

clk: Cung cấp xung nhịp cho toàn bộ hệ thống.

rst_n: Tín hiệu reset (mức thấp để reset).

dht11: Chân kết nối cảm biến DHT11, hoạt động theo giao tiếp 1-wire.

lcd_rs, lcd_e, lcd_db: Các tín hiệu điều khiển màn hình LCD 16x2.

led_hum, led_temp: LED hiển thị giá trị đo được từ DHT11

2. Khai báo các tín hiệu trung gian

```

10 );
11
12     wire [31:0] data_valid; // Thêm khai báo này
13     wire [15:0] temperature;
14     wire [15:0] humidity;
15     wire clk_1MHz_out;

```

data_valid: Dữ liệu 32-bit từ cảm biến DHT11 (16 bit cao là độ ẩm, 16 bit thấp là nhiệt độ).

temperature: Lưu giá trị nhiệt độ sau khi tách từ data_valid.

humidity: Lưu giá trị độ ẩm sau khi tách từ data_valid.

clk_1MHz_out: Tín hiệu xung nhịp 1MHz được tạo ra để điều khiển DHT11.

Tạo xung nhịp 1MHz

```

16
17     Clock_1MHz CLK1MHZ(clk, rst_n, Clock_1MHz_out);
18

```

Chức năng:

- Tạo tín hiệu xung nhịp 1MHz từ tín hiệu clk của hệ thống.
- Cảm biến DHT11 hoạt động với thời gian đáp ứng chậm, cần tần số thấp hơn so với xung nhịp hệ thống.

Giao tiếp với cảm biến DHT11

4. Giao tiếp với cảm biến DHT11

```

24 DHT11_Controller dht11_ctrl (
25     .clk_1M(Clock_1MHz_out),
26     .rst_n(rst_n),
27     .dht11(dht11),
28     .data_valid(data_valid) // Kết nối đúng tín hiệu
29 );

```

Chức năng:

- Điều khiển cảm biến DHT11 để lấy dữ liệu nhiệt độ và độ ẩm.
- Khi cảm biến hoàn tất quá trình đo, dữ liệu sẽ được lưu vào data_valid.
- Sử dụng xung nhịp 1MHz để đảm bảo đúng thời gian giao tiếp.

Xử lý dữ liệu từ DHT11

```

25 L
26 assign humidity = data_valid[31:16];
27 assign temperature = data_valid[15:0];
28

```

Tách dữ liệu data_valid thành hai phần:

- Độ ẩm: data_valid[31:16] (16 bit cao).
- Nhiệt độ: data_valid[15:0] (16 bit thấp).

```

28
29 LCD_Controller lcd_ctrl (
30     .clk(clk),
31     .temperature(temperature),
32     .humidity(humidity),
33     .lcd_rs(lcd_rs),
34     .lcd_e(lcd_e),
35     .lcd_db(lcd_db)
36 );
37

```

Nhận giá trị nhiệt độ và độ ẩm từ DHT11_Controller.

Gửi dữ liệu đến màn hình LCD 16x2 để hiển thị.

Giao tiếp với LCD thông qua giao tiếp 8-bit.

Xuất dữ liệu ra LED để kiểm tra

```

36 |      //
37 |
38 |      // Xuất dữ liệu nhiệt độ ra LED để kiểm tra
39 |      assign led_hum = data_valid[31:24];
40 |      assign led_temp = data_valid[13:8];
41 |
42 |      endmodule
43 |

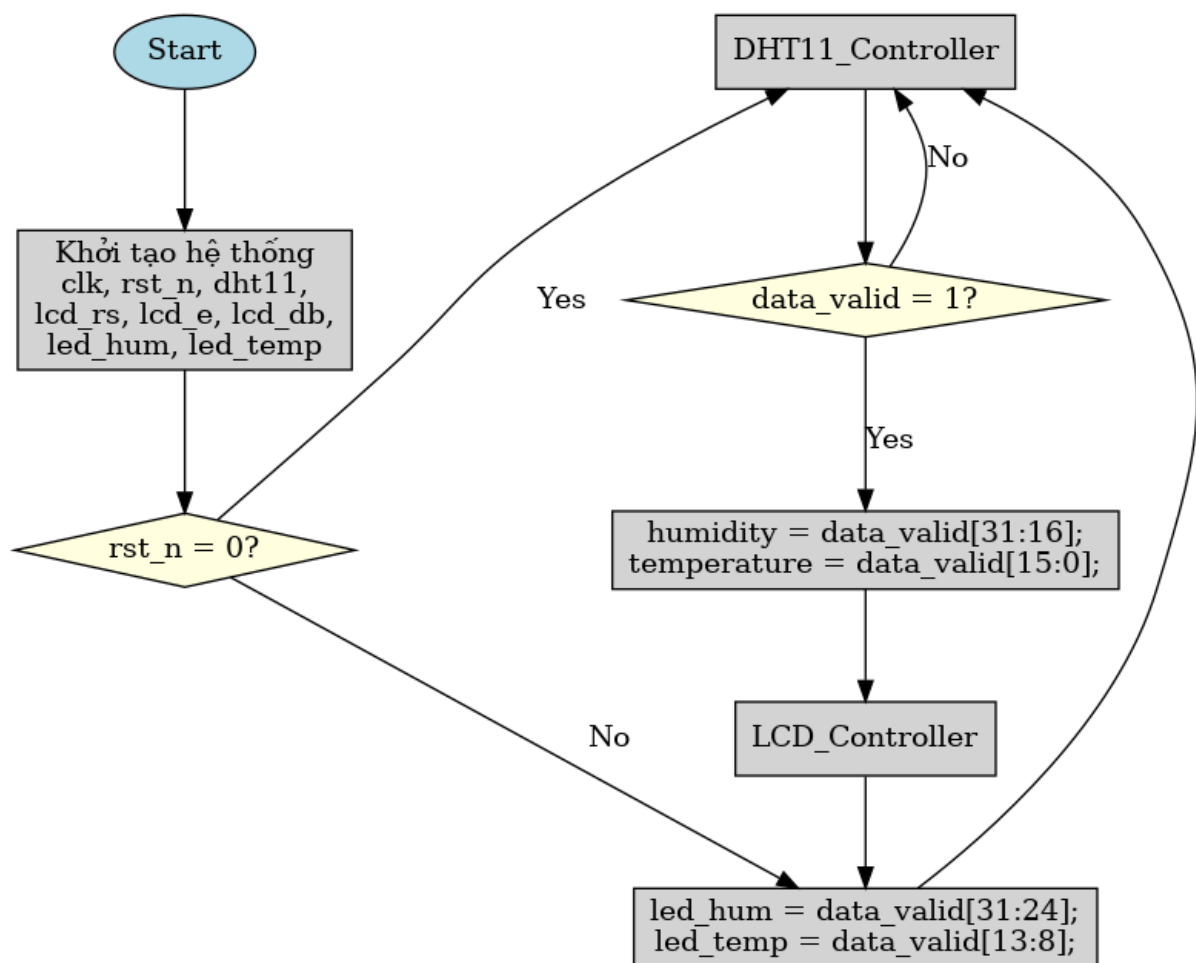
```

LED hiển thị độ ẩm: Lấy 8 bit cao của độ ẩm (data_valid[31:24]), tức là phần nguyên của giá trị độ ẩm.

LED hiển thị nhiệt độ: Lấy 6 bit từ data_valid[13:8] để thể hiện nhiệt độ.

LED sẽ sáng theo giá trị đo được, giúp kiểm tra dữ liệu mà không cần màn hình LCD.

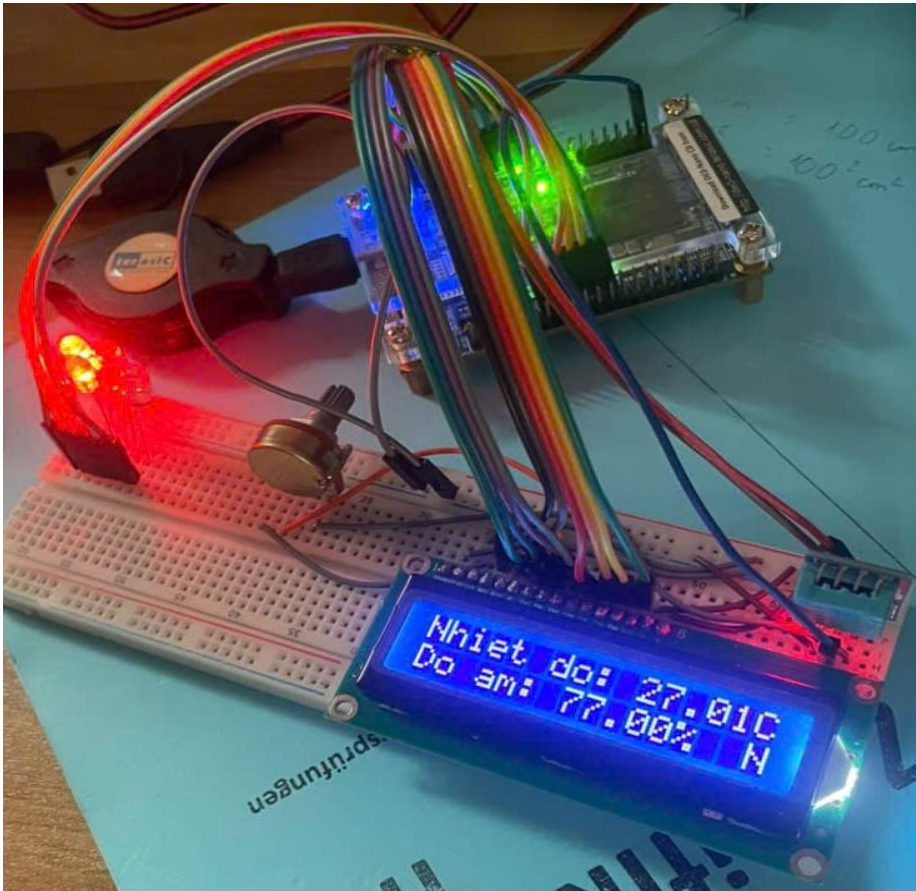
Lưu đồ thuật toán



Hình 20:: Lưu đồ thuật toán

CHƯƠNG 3: GIẢI THÍCH VÀ KẾT QUẢ

1. Kết quả thực nghiệm



2. Kết luận

Về kiến thức :

- ❖ Đã nghiên cứu và tìm hiểu về nguyên lý hoạt động của cảm biến nhiệt độ và độ ẩm.
- ❖ Hiểu và áp dụng ngôn ngữ lập trình phân cứng Verilog HDL vào thiết kế hệ thống đo lường.
- ❖ Nắm vững cách sử dụng FPGA trong các ứng dụng thực tiễn, đặc biệt là trong việc điều khiển và thu thập dữ liệu từ cảm biến

Về thực nghiệm:

- ❖ Chế tạo thành công thiết bị đo nhiệt độ và độ ẩm sử dụng KIT FPGA DE0 Nano, kết hợp với cảm biến DHT11 và màn hình LCD. 73
- ❖ Kiểm thử hệ thống trên môi trường thực tế, đánh giá độ chính xác của dữ liệu thu thập được.

- ❖ Nâng cao kỹ năng lập trình phần cứng, phân tích tín hiệu và xử lý dữ liệu trên FPGA.
- ❖ Cũng cố kỹ năng vận hành thiết bị đo, sử dụng phần mềm Quartus II để mô phỏng và lập trình hệ thống.

KẾT LUẬN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN

Tóm lược thành tựu:

Dự án "Measuring Device" đã chứng minh thành công khả năng ứng dụng FPGA trong việc xây dựng một hệ thống đo lường môi trường toàn diện. Thông qua việc sử dụng ngôn ngữ Verilog HDL, nhóm đã đạt được những kết quả đáng kể:

Nền tảng kiến trúc vững chắc: Dự án đã củng cố và mở rộng kiến thức về lập trình FPGA, đặc biệt là trong việc thiết kế và triển khai các ứng dụng thời gian thực.

Hệ thống đo lường chính xác: Hệ thống đã chứng minh khả năng đo lường chính xác các thông số nhiệt độ, độ ẩm và áp suất từ các cảm biến ngoại vi, đáp ứng yêu cầu về độ tin cậy trong các ứng dụng thực tế.

Giao diện người dùng trực quan: Việc xây dựng giao diện người dùng đơn giản nhưng hiệu quả đã tạo điều kiện thuận lợi cho việc theo dõi và kiểm soát dữ liệu đo được.

Định hướng phát triển chiến lược:

Để tối đa hóa tiềm năng và mở rộng phạm vi ứng dụng của dự án, các định hướng phát triển sau đây được đề xuất:

Mở rộng khả năng tương thích và linh hoạt: Thiết kế hệ thống với khả năng tích hợp linh hoạt, cho phép kết nối và xử lý dữ liệu từ nhiều loại cảm biến khác nhau, từ đó mở rộng phạm vi ứng dụng trong các môi trường đa dạng.

Tối ưu hóa hiệu suất hệ thống: Tập trung vào việc tối ưu hóa mã nguồn và cấu trúc phần cứng để nâng cao hiệu suất xử lý, giảm thiểu độ trễ và tiết kiệm tài nguyên FPGA, đảm bảo hệ thống hoạt động ổn định và hiệu quả trong các điều kiện khắc nghiệt.

Tích hợp khả năng kết nối và lưu trữ dữ liệu: Phát triển khả năng kết nối mạng (Ethernet, Wi-Fi) để truyền tải dữ liệu đo được đến các hệ thống quản lý trung tâm hoặc đám mây. Đồng thời, tích hợp khả năng lưu trữ dữ liệu trên các thiết bị ngoại vi (thẻ SD, USB) để phục vụ cho việc phân tích và báo cáo.

Nâng cao tính bảo mật dữ liệu: Triển khai các biện pháp bảo mật mạnh mẽ để bảo vệ dữ liệu đo được khỏi các truy cập trái phép và đảm bảo tính toàn vẹn của hệ thống trong môi trường kết nối.

Phát triển các tính năng thông minh: Nghiên cứu và tích hợp các thuật toán học máy và trí tuệ nhân tạo để phát triển các tính năng tự động điều chỉnh

và dự báo, từ đó nâng cao giá trị và tính ứng dụng của hệ thống trong các lĩnh vực như nông nghiệp thông minh, quản lý năng lượng và giám sát môi trường.

Kết luận:

Dự án "Measuring Device" đã đặt nền móng vững chắc cho việc phát triển các hệ thống đo lường thông minh dựa trên nền tảng FPGA. Việc tiếp tục đầu tư vào các định hướng phát triển đã đề ra sẽ mở ra nhiều cơ hội ứng dụng tiềm năng và góp phần vào sự phát triển của công nghệ đo lường hiện đại.

TÀI LIỆU THAM KHẢO

- [1].<https://www.intel.vn/content/www/vn/vi/support/programmable/support-resources/fpga-training/getting-started.html>
- [2] Palnitkar, S. (1996). Verilog HDL.
- [3] Ciletti, M.D. (2004). Advanced Digital Design With the Verilog HDL.
- [4] Donzellini, G., Oneto, L., Ponta, D. & Anguita, D. (2019). Introduction to Digital Systems Design.
- [5] DE0-Nano User Manual
- [6].<https://www.zun.vn/tai-lieu/tim-hieu-ve-phan-mem-quartus-ii-33739/> (Online
- [7] <https://www.programmersought.com/article/90045264630/>
- [8] Sensors Modules Lcd 16x2 Display Module | Sensors Modules
- [9] GitHub - BharatVigyan/verilog: verilog codes for various sensor for interface with FPGA boards
- [10] GitHub - EZIOPQR/fpga_dht11
- [11] vhdl - Why even use std_logic instead of std_ulogic for implantation purposes? - Stack Overflow
- [12] Xilinx FPGA Architecture - GeeksforGeeks